



KNOWCAST: Domain-knowledge-integrated deep learning for metro OD forecasting and path flow estimation

Hao Zheng^a, Han Zheng^{a,*}, George Giannopoulos^c

^a Beijing Jiaotong University, No. 3 Shangyuncun, Haidian District, Beijing, 100044, China

^b Tsinghua University, No. 30 Shuangqing Road, Haidian District, Beijing, 100084, China

^c Aristotle University of Thessaloniki (emeritus), Thessaloniki, 54124, Greece

ARTICLE INFO

Keywords:

Metro system
OD forecasting
Path flow estimation
Knowledge-informed deep learning
Graph attention networks

ABSTRACT

In metro operations, Origin-Destination (OD) flows and path flows are critical for transportation planning, but their accurate short-term forecasting and estimation remain challenging due to the complex influencing factors that affect passenger behavior. In addition, due to data collection challenges, previous research has generally prioritized short-term OD flow forecasting over path flow estimation. To address these challenges, this paper presents a domain-knowledge-integrated framework for jointly forecasting short-term network-level OD flows and inferring path flows, with path-flow inference directly coupled to the predicted OD flows through an embedded assignment mechanism. The framework systematically integrates transportation domain knowledge of trip generation, trip distribution, and traffic assignment into a sequence-to-sequence architecture. This integration enables the model to analyze and incorporate these influencing factors into its predictions. Furthermore, the proposed adaptive training mechanism progressively refines the influence of these factors by integrating domain-knowledge-based components directly into the sequence-to-sequence architecture, with key parameters initialized using domain knowledge to provide an effective starting point for subsequent optimization. The direct coupling between OD forecasts and inferred path-flow estimates is promoted by leveraging intermediate features that encode path-level information. Generated by the domain-knowledge-based components during their joint optimization with the main network, these features are used to produce path-flow estimates that remain structurally linked to the OD forecasts. The experimental results demonstrate the effectiveness of the proposed method. By analyzing the optimization trajectories of the domain-knowledge-derived parameters, this study shows how they evolve during training and how the deep learning process refines the initial parameters derived from classical models to improve predictive performance. The source code used in this study is publicly available at the GitHub repository: <https://github.com/scholarhaozheng/KNOWCAST>.

1. Introduction

As the backbone of modern urban transportation, metro systems require continuous and adaptive operational management to maintain service reliability under fluctuating demand and service conditions. These fluctuations arise from both routine high-variability

* Corresponding author.

E-mail addresses: zhengh22@mails.tsinghua.edu.cn (H. Zheng), han_zheng@bjtu.edu.cn (H. Zheng), ggian@academyofathens.gr (G. Giannopoulos).

Giannopoulos).

URL: <http://en.trans.bjtu.edu.cn/faculty/52vYPdmZmW7AlVvkq.html> (H. Zheng), <http://giannopoulosgeorge1.academia.edu/> (G. Giannopoulos).

<https://doi.org/10.1016/j.trc.2026.105710>

Received 15 July 2025; Received in revised form 17 March 2026; Accepted 15 April 2026

Available online 15 May 2026

0968-090X/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

events, such as special events (Li et al., 2017a; Lv et al., 2024) (e.g., sports games and holidays) that alter passenger flow, and significant service disruptions. The latter encompass weather events (Mitsakis et al., 2014) (e.g., storms, very high temperatures or droughts), natural disasters (Zhang et al., 2024) (e.g., fires, epidemics, flooding), and mechanical breakdowns (Zou et al., 2024) (e.g., power outages and signal failures). Effectively responding to these conditions through proactive operational strategies, such as crowd control, shutdowns, cancellations, or frequency adjustments, depends on accurate flow forecasting and estimation. In practice, such quantified flow insights are needed at two levels. Forecasts of Origin-Destination (OD) flows provide a network-wide view, which is crucial for strategic decisions, such as adjusting service schedules. Estimates of path flows provide a granular perspective essential for capacity allocation. While station-level forecasts signal where crowding occurs, path-flow estimates disentangle this aggregated demand to reveal specific route usage. This enables path- and line-level interventions, such as adjusting line-specific service frequencies.

While there is a clear operational need for short-term OD forecasting and path-flow estimation, current research and practice face four gaps: two practical challenges related to data observability and influencing-factor integration, and two methodological or structural limitations in existing modeling approaches.

The first gap is a practical observability challenge arising from the limitations of Automated Fare Collection (AFC) records, the primary data source for metro demand analysis. AFC records capture passengers' origins, destinations, and transaction times, but they do not record the specific routes or trajectories chosen within the network. Emerging sensing technologies can provide supplementary information, but their use remains constrained in complex underground environments. Wi-Fi and Bluetooth probes, for example, may not reliably track cross-station trajectories because of MAC address randomization and signal attenuation. Train load weighing mechanisms and automated door counters provide only indirect approximations of carriage-level flows, since they rely on static weight assumptions and are vulnerable to cumulative counting errors. As a result, AFC data can identify entry–exit transactions but cannot directly observe the realized en-route trajectory or path label in a multi-transfer network. Segment-level flows therefore cannot be directly observed from AFC records. This observability barrier makes joint OD and path-flow estimation an under-determined problem that is difficult to resolve using purely data-driven methods alone. Additional structural assumptions are therefore needed, particularly traffic-assignment principles that regularize the solution space and link OD demand to plausible route choices.

The second gap concerns the practical difficulty of modeling passenger-flow dynamics, particularly in analyzing and integrating influencing factors into predictive models. In addition to the large-scale AFC datasets generated by metro systems, researchers now have access to external records that describe relevant influencing factors, such as land use characteristics, catchment population, operational adjustments, and weather conditions. However, data availability alone does not guarantee high forecasting accuracy. Simply ingesting these datasets and influencing factors as flat input features is often insufficient; an effective model requires a specialized architecture that can explicitly represent and learn how these factors shape flow dynamics.

The remaining two gaps concern methodological and structural limitations in existing deep learning approaches.

The third gap is that current deep learning models often struggle to capture the effects of dynamic influencing factors without the integration of transportation domain knowledge. The core of this issue is methodological: existing approaches lack explicit mechanisms to map these influencing factors onto behaviorally meaningful determinants (e.g., travel-time perception and transfer penalties). Rather than using transportation domain knowledge to encode transportation behavioral rules that describe how influencing factors (e.g., weather, service disruptions, or operational adjustments) may affect passenger travel behavior, a common approach is to concatenate these factors as feature vectors (Liang et al., 2019; Lu et al., 2021). As a result, the network must infer these relationships implicitly from the training data, which limits interpretability and may reduce its ability to generalize when operational conditions shift.

The fourth gap lies in the absence of unified models for simultaneous OD-flow prediction and path-flow estimation. Most state-of-the-art spatiotemporal frameworks (Gugaliya et al., 2021; Liu et al., 2019c) primarily focus on forecasting the OD matrix, relegating traffic assignment to a subsequent post-processing step. Such decoupled approaches may not fully capture the dynamic interdependencies between OD demand and network propagation, and thus risk generating path-flow estimates that are operationally inconsistent with the evolving demand patterns. Overcoming this limitation requires a joint approach that structurally models the interdependencies between OD and path flows based on traffic-assignment principles, thereby improving both model consistency and predictive performance.

This leads to the central research question of this paper: How can we develop a unified deep learning framework that leverages transportation domain knowledge to forecast metro OD flows and simultaneously infer path flows without path-level ground-truth labels, while maintaining interpretability and adaptability to dynamic influencing factors?

To bridge these identified gaps, this paper proposes a domain-knowledge-integrated framework, KNOWCAST (Knowledge-Integrated OD Forecasting and Path-Flow Estimation), for joint metro network-level OD-flow forecasting and path-flow estimation. Accordingly, we make the following contributions.

1. To effectively integrate influencing factors into deep learning models, we apply domain knowledge from transportation planning, including trip generation, trip distribution, and traffic assignment. For trip generation, we analyze station-level impacts, evaluating how local socio-demographic profiles and major events influence the passenger volume originating from individual stations. For trip distribution, we examine how influencing factors (e.g., line suspensions or service-frequency changes) affect inter-station travel patterns by altering travel impedance and passenger flows between station pairs. For traffic assignment, we structurally model the propagation of demand onto network segments, capturing how adjustments to network topology and service frequency dynamically redirect passenger route choices and alter section-level flows. By translating these influencing factors into network

impacts, we leverage them for several model enhancements: constructing domain-knowledge-augmented feature representations, introducing a domain-knowledge-guided loss term, and developing a knowledge-informed architecture together with domain-knowledge-based parameter initialization, thereby enabling the model to learn from these factors more efficiently.

2. To jointly forecast network-level OD flows and estimate path flows, we develop a unified, knowledge-integrated deep learning framework. This architecture integrates knowledge-based components for trip generation, trip distribution, and traffic assignment with a deep learning predictor that supports both spatiotemporal OD forecasting and assignment-based path-flow inference. Specifically, intermediate representations generated for OD prediction are directly fed into an embedded, differentiable assignment layer (formulated as a trainable Logit layer). Unlike methods that treat traffic assignment as an isolated downstream task after OD forecasting, this design explicitly models the structural dependency between predicted OD demand and inferred path-flow estimates within a single end-to-end model. This design therefore enables path-flow inference without requiring path-level ground-truth labels, while keeping the inferred path-flow estimates directly coupled to the predicted OD demand.
3. To enhance the practical adaptability and interpretability of our model, we integrate domain knowledge at both the architectural and parameter-initialization levels. We incorporate knowledge-based components into the model's architecture and initialize selected trainable parameters with knowledge-derived values. This enables the model to begin with a domain-knowledge-informed foundation while maintaining the flexibility to adjust these parameters in response to real-world data. The post-training trajectories of the knowledge-derived parameters can be explicitly analyzed, showing how the model fine-tunes these behavioral parameters to better align with the observed network-level OD dynamics, even in the absence of path-level labels.

This research enables a more cohesive, interpretable, and practically applicable integration of transportation domain knowledge and advanced deep learning techniques for network-level analysis of passenger flows.

2. Literature review

This section reviews existing methodologies for metro passenger flow forecasting. Table 1 provides a structured overview, categorizing methods from classical transportation models to modern deep learning approaches and highlighting their treatment of the challenges that motivate this work: the integration of influencing factors and the consistent forecasting of OD flows and estimation of path flows.

As a classic framework in transportation planning, the four-step model offers a theoretically grounded modeling process built on transportation domain knowledge, benefits from established industry standards, mature calibration techniques, and countless successful road-network applications. Its principles are also applied to the metro domain, where elements of the model are utilized to inform planning decisions, even if the entire four-step process is not always implemented in its classical, sequential form (Mo et al., 2020a; Peng et al., 2024). However, when applied to short-term metro OD flow prediction, the model is not well-equipped to effectively leverage the real-time OD data generated by the railway system. It is unable to learn directly from the patterns within these data, nor can it efficiently utilize relevant information concerning dynamic operational changes, such as variations in train service frequencies or network disruptions (e.g., station or line section closures). Consequently, forecasts derived purely from the four-step model often exhibit lower accuracy for short-term OD flow prediction compared to methods capable of directly utilizing this detailed information. Thus, while classical transportation models offer foundational domain knowledge, their practical utility in predicting metro passenger flow is significantly constrained.

To overcome the limitations of classical, domain-knowledge-based methods, statistical (parameterized) models emerged. These models predict future OD volumes by fitting parameters directly to historical data, without embedding explicit knowledge of network topology, service schedule, or passenger behavior, and do not rely on externally provided domain knowledge or information from traffic surveys. The Historical Average (HA) method simply computes the mean volume for each time interval and assumes consistent temporal patterns, providing a straightforward implementation, but it fails to adapt to sudden demand shifts (Smith and Demetsky, 1997). ARIMA models use auto-regressive and moving-average terms to capture linear temporal dependencies, but are constrained by assumptions of stationarity, and often face challenges when adapting to abrupt or nonlinear variations (Ahmed and Cook, 1979; Anvari et al., 2016; Ma et al., 2017; Vlahogianni et al., 2014). Least Squares methods (LS) formulate OD matrix estimation as a regression problem constrained by link counts and assignment models (Yao et al., 2016). Although these statistical methods can provide baseline OD forecasts, they can be constrained by their simplified assumptions. They primarily attempt to extract historical patterns, but real-world metro passenger flow is frequently influenced by real-time, non-periodic influencing factors, which are common in daily operations. As these models lack mechanisms to account for such disruptions, their practical accuracy is often compromised.

Machine-learning approaches such as Bayesian networks (BN), Backpropagation Neural Network (BPNN), and Support Vector Machines (SVMs) have been deployed to forecast metro OD flows by learning complex, non-linear relationships directly from the data, yet each technique comes with distinctive trade-offs. Bayesian networks encode causal dependencies (e.g., headways, station connectivity, weather) in a directed acyclic graph and update OD probabilities via Bayes' rule, allowing heterogeneous evidence fusion and explicit uncertainty bounds (Blume et al., 2022; Ma et al., 2013; Roos et al., 2017). However, when the number of stations and features increases, structure learning and exact inference become NP-hard, so analysts must rely on strong independence assumptions or approximate sampling methods. These shortcuts often reduce accuracy during peak-hour demand surges. Back-propagation neural networks iteratively adjust weights through gradient back-propagation, capturing the non-linear interplay among timetable changes, boarding counts, and spatial lags (Roos et al., 2016; Wei and Chen, 2012; Zhou et al., 2016). They can introduce impact features into training without heavy feature engineering. However, a plain feed-forward network has no explicit temporal memory, so it struggles to

Table 1

Categorization of passenger flow forecasting methods.

Specific Technique	Analysis of Limitations	Representative Citations
Classical Transportation Models		
Four-Step Model	Factor Integration: Theory-grounded but not adaptive to real-time data streams.	Mayer (1961), Mo et al. (2020a), Peng et al. (2024)
Classical Optimization		Tsekeris et al. (2007)
Statistical (Parameterized) Models		
Historical Average (HA)	Factor Integration: Not explicitly modeled; assumes stationary patterns and struggles with dynamic disruptions. Prediction Target: Almost exclusively OD flow only.	Smith and Demetsky (1997)
ARIMA		Anvari et al. (2016), Ma et al. (2017), Vlahogianni et al. (2014), Ahmed and Cook (1979)
Kalman Filtering		Jiao et al. (2016), Xiong et al. (2020), Gazis and Knapp (1971), Grindey et al. (n.a.), Guo et al. (2014)
State-Space Models		Yao et al. (2015), Zhi-jie et al. (2017)
Least Squares (LS)		Yao et al. (2016)
Iterative Proportional Fitting		Galliani et al. (2024)
Classic Machine Learning		
Bayesian Networks (BN)	Factor Integration: Handled as input features without structural guidance. Prediction Target: Overwhelmingly OD flow only.	Blume et al. (2022), Ma et al. (2013), Roos et al. (2017)
BP Neural Network (BPNN)		Roos et al. (2016), Wei and Chen (2012), Zhou et al. (2016)
Support Vector Machines (SVMs)		Sapankevych and Sankar (2009), Sun et al. (2015), Wang et al. (2018)
Spatiotemporal Deep Learning		
LSTM	Factor Integration: Predominantly concatenates influencing factors as raw feature vectors, lacking theoretical guidance. Prediction Target: Overwhelming focus on OD flow prediction, neglecting joint and consistent path flow estimates.	Cho et al. (2014), Hochreiter and Schmidhuber (1997), Jiang et al. (2019), Liu et al. (2023, 2019c), Noursalehi et al. (2022), Toqué et al. (2016), Wang et al. (2020, 2019), Xi et al. (2018), Zhang et al. (2019a), Ma et al. (2018)
CNN		Krizhevsky et al. (2017), Shi et al. (2015), Zhang et al. (2021)
GCN		Gugaliya et al. (2021), He et al. (2022), Jiang et al. (2022), Liu et al. (2020), Wang et al. (2020, 2019), Xiong et al. (2020), Yang et al. (2022), Liu et al. (2023), Guo et al.
GRU		Liu et al. (2023), Sha et al. (2020)
Encoder-Decoder		Ke et al. (2021)
Emerging Knowledge-Guided Models		
Knowledge-Guided Architectures	Factor Integration: Begins to use domain knowledge to structure model inputs or logic. Prediction Target: Primarily OD flow.	Zhang et al. (2023b), Zou et al. (2024), Jin et al. (2025)

track long-range demand cycles. Support Vector Machines employ structural-risk minimization to find a maximum-margin hyperplane that links features such as lagged boardings, weather, and special events to OD demand (Sapankevych and Sankar, 2009; Sun et al., 2015; Wang et al., 2018). Their limitations become more pronounced on large networks: the computational effort required to solve the underlying quadratic program scales super-linearly with the number of OD pairs, kernel and hyper-parameter tuning require exhaustive search, and the default formulation provides only deterministic point estimates. Collectively, despite their advantages over simple statistical baselines, a challenge for these classical machine learning models is processing the spatiotemporal graph structure of metro OD data. Metro OD data is shaped by both its evolution over time (temporal dependencies) and the physical network on which it exists (spatial-topological dependencies). As these models were not primarily designed for this context, they may struggle to simultaneously capture both the temporal and spatial dimensions, and their computational complexity can become a hurdle for large-scale applications.

The rise of deep learning has introduced a new paradigm for tackling prediction problems. Deep learning architectures are comparatively better suited to utilize the spatiotemporal data generated by metro systems. By representing the metro network as a graph with stations as nodes, Graph Neural Networks (GNNs) and Graph Convolutional Networks (GCNs) capture the spatial diffusion of passenger flow across the metro network topology (Gugaliya et al., 2021; He et al., 2022; Jiang et al., 2022; Liu et al., 2020; Wang et al., 2020, 2019; Xiong et al., 2020; Yang et al., 2022). To model temporal dynamics, recurrent architectures such as LSTM or GRU capture long-range headway and boarding dependencies (Cho et al., 2014; Hochreiter and Schmidhuber, 1997; Jiang et al., 2019; Liu et al., 2023, 2019c; Noursalehi et al., 2022; Sha et al., 2020; Toqué et al., 2016; Wang et al., 2020, 2019; Xi et al., 2018; Zhang et al., 2019a) in ridership patterns. GNNs and GCNs are often combined with recurrent architectures to capture the spatiotemporal relationships of metro passenger flow. For example, hybrid models such as the Semantic-Augmented ST-GCN (Jin et al., 2025) or DeepPF (Liu et al., 2019c) can process raw AFC data and timetables, allowing them to learn how schedule changes, station topology, and

passenger behavior clusters interact over time. Recent studies have increasingly leveraged heterogeneous data sources. Galliani et al. (2024) proposes a dynamic OD estimation pipeline that fuses ticket and subscription sales with automated passenger-count data using an Iterative Proportional Fitting algorithm, yielding weekly OD matrices for the Trenord railway network. Zou et al. (2024) develops a modular real-time OD-flow prediction model by combining convolutional LSTM cells, self-attention layers, and enhanced graph convolutions that are augmented with impact features like weather to capture rapid spatiotemporal disruptions during operational incidents.

However, despite the impressive predictive capabilities of deep learning models, a closer examination reveals a limitation in how these models incorporate influencing factors. Real-world metro operations are constantly influenced by a variety of influencing factors, such as weather conditions (Guo et al., 2020), air quality (Zhang et al., 2020), day of the week (Zhang et al., 2017), holidays (Chen et al., 2019), and events (Chen et al., 2019), which require adaptive management strategies. These passenger flow management strategies can directly or indirectly influence the metro service network, as well as change how passengers enter and move through the system. The effectiveness of a predictive model therefore depends on whether it can remain reliable during the disruptions that commonly occur in daily operations. Performance under incident-free settings alone is not sufficient. To account for influencing factors affecting OD flow distribution, previous studies have explored different methods to integrate these factors into deep learning-based passenger OD flow forecasting models. The predominant method for incorporating these factors is to embed them as unstructured augmented feature vectors and concatenate them with other input, either with the primary input tensor (Liang et al., 2019; Liu et al., 2019a; Lu et al., 2021; Yao et al., 2019), or with the encoded spatiotemporal features before the final prediction layer (Zhang et al., 2020, 2017, 2018, 2019b). This approach, while straightforward in providing the model with raw information, generally does not explicitly incorporate transportation domain knowledge to define how these influencing factors logically shape passenger travel behavior, such as trip generation or distribution patterns. Consequently, the model must infer these complex relationships from correlations in the training data and the directly incorporated, unanalyzed influencing factors alone. This inference process is often challenging. A few studies have begun to address this gap by using domain knowledge to guide the analysis and integration of influencing factors. For example, Zhang et al. (2023b) employs a passenger travel path choice model to calculate the passenger flow volumes on a specific route section. Then it calculates the relative importance of OD pairs and incorporates this importance into the loss function to guide the optimization direction. However, these knowledge-guided approaches remain the exception.

Beyond the challenge of knowledge integration, the fundamental limitation is that current models are primarily designed to predict overall passenger demand (OD flows), and in doing so, largely neglect the prediction of how that traffic will spread out across specific routes (path flows). As summarized in Table 1, most advanced deep learning models, including the spatiotemporal frameworks cited earlier, define their primary objective as predicting the OD matrix. They largely overlook the task of producing path-flow estimates that remain directly coupled to the OD results. If these two tasks are separated, a potential limitation of this separation is that it may not fully capture the strong inter-dependencies between OD and path flows. The challenge of ensuring their consistency is explicitly recognized in transportation science. For instance, some researchers have focused on the joint estimation of OD matrices and path flows to find a mutually consistent solution, often using classical optimization (Tsekeris et al., 2007). However, these methods are often distinct from the end-to-end deep learning paradigm. Within the deep learning context, where models are trained directly on data, forecasting OD flows and estimating path flows with separate, uncoordinated models creates the risk of generating logically inconsistent and operationally unreliable outputs. Therefore, a critical research gap exists: the absence of a unified deep learning framework capable of jointly forecasting OD flows and inferring path flows while keeping the two outputs directly coupled within a single model.

This review thus confirms the two primary methodological gaps identified in Section 1. First, we have seen that while deep learning models are powerful, they primarily treat influencing factors as unstructured inputs, failing to leverage transportation domain knowledge for more robust and interpretable predictions. Second, the literature shows a clear and persistent focus on OD flow forecasting, with a significant lack of unified frameworks that can ensure consistency between OD predictions and path flow estimations. Our work is therefore positioned to directly address these limitations by proposing a knowledge-integrated architecture for joint forecasting and estimation.

3. Framework for integrating domain knowledge into forecasting models

The KNOWCAST framework integrates domain knowledge from classical transportation models (including trip generation, trip distribution, and traffic assignment) into its deep learning architecture through its core component, the Knowledge-Integrated Traffic Predictor (KITP). These KITPs serve as the building blocks for both the encoder and decoder in KNOWCAST's sequence-to-sequence (Seq2Seq) architecture. Embedding KITP units throughout the encoder-decoder structure allows KNOWCAST to leverage intermediate representations and outputs generated during OD forecasting, such as path-choice probabilities, to infer path flows even when direct path-flow observations are unavailable.

3.1. The integration of domain-knowledge and deep learning model

To provide a clear roadmap before discussing the technical complexities, Fig. 1 outlines the high-level logic of our framework. It demonstrates how transportation domain knowledge is translated into Knowledge Integrators that steer the deep learning engine toward consistent multi-level predictions. Building on this overview, we introduce the conceptual basis for integrating transportation domain knowledge with deep learning models to handle the influencing factors. We define domain knowledge as structured, specialized information that can take various forms, such as model coefficients that quantify the importance of factors, functional forms

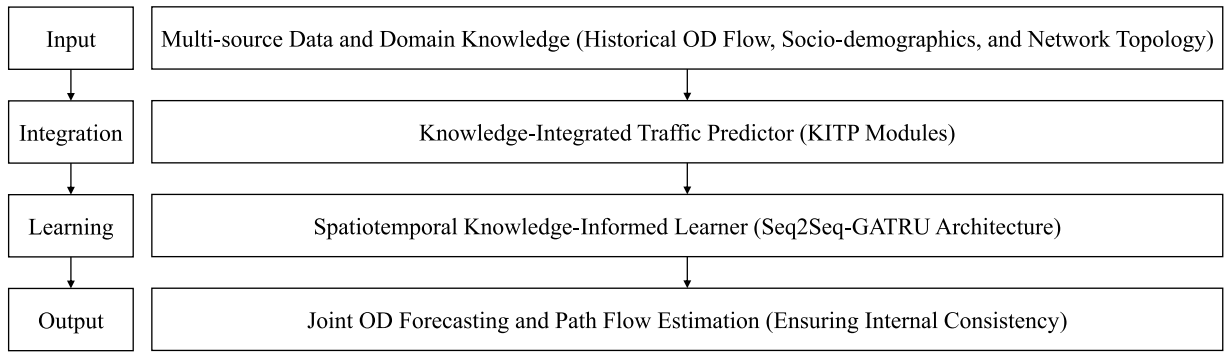


Fig. 1. Conceptual roadmap of the KNOWCAST framework. It illustrates the streamlined flow from raw data and domain knowledge to joint and consistent network-level OD forecasting and path flow estimation.

that express behavioral rules as mathematical equations, and network topologies that represent the structural relationships between nodes.

Expanding upon the initial roadmap, Fig. 2 presents the detailed methodology for knowledge integration. In deep learning, a model M comprises several components: a dataset D , a network architecture π , a loss function $\mathcal{L}$, and a set of parameters θ . The training process is to learn the optimal parameters θ for a model M , which is achieved by using a learner L to minimize a loss function $\mathcal{L}$ on the given dataset D (Dash et al., 2022). Within this context, we define *knowledge integration* as the process of translating problem-specific domain knowledge into targeted modifications of one or more of these components: the data D , the architecture π , the parameters θ , or the loss function $\mathcal{L}$, and integrating them into the model. This translation and integration are performed by what we term a *Knowledge Integrator* (as indicated by ①, ②, ③, ④, and ⑤).

Our four-layered methodology for knowledge integration is detailed as follows:

- (a) Raw Data: Presents the initial model inputs, including historical OD data and various influencing factors.
- (b) Domain Knowledge: Represents the repository of domain knowledge, including transportation principles and parameters.
- (c) Knowledge Integrator: Employing domain knowledge to analyze the impacts of influencing factors on the metro system and translate these impacts into components that can be directly integrated into deep learning models. We formalize this into four primary methods of knowledge integration:
 - (i) **Method 1: Data Transformation:** Wherein domain knowledge is analyzed to process relevant influencing factors and extend the original data D with domain-knowledge-integrated features, produce domain-knowledge-enriched data D' , which serves as the actual input to the model.
 - (ii) **Method 2: Loss Function Modification:** Wherein knowledge-informed terms are integrated into the original loss function $\mathcal{L}$, forming an enhanced loss function $\mathcal{L}'$ that incorporates domain-theoretical guidance during model optimization.
 - (iii) **Method 3: Architecture Extension:** Wherein domain knowledge extends the model's architecture π into π' by incorporating a knowledge-based component.
 - (iv) **Method 4: Parameter Initialization:** Wherein domain knowledge provides initial values for a subset of the knowledge-based component's parameters θ , allowing the model to begin with a transportation theory-based foundation and optimize these parameters dynamically during training. This process creates domain-knowledge-informed parameters θ' , which are subsequently made trainable.
- (d) Learner: Receives the knowledge-integrated data (D'), parameters (θ'), extended architecture (π'), and the knowledge-informed loss function ($\mathcal{L}'$), and is trained to generate predictions.

We consider influencing factors on the metro system at three planning stages: trip generation, trip distribution, and traffic assignment. We then outline the methodology for using Knowledge Integrators to transform the knowledge from each stage into structured, model-integrable components for deep learning.

To account for the impact of socio-demographic features and temporal attributes, which are the influencing factors perturbing station inflows and outflows, we employ a Knowledge Integrator to aggregate them to each station's Traffic Analysis Zone (TAZ) and predict the time-varying trip generation and attraction for each TAZ. The difference between these predictions and the OD flow forecasts from the main model (which are summed by station) acts as a loss function term. This term utilizes domain knowledge from the trip generation phase to direct the main model's training.

Influencing factors, including operational adjustments (e.g., planned headway changes, station closures, and line or section suspensions) and incident-related service disruptions or response measures (e.g., service cancellations, emergency station closures), can alter generalized inter-station impedance and thereby reshape station-to-station travel patterns. We employ a Knowledge Integrator to encode these impacts as time-varying inputs to a gravity model, and then run a trip-distribution algorithm to compute a coarse estimate of OD proportions. The derived OD proportions are appended to the OD feature vector to improve sensitivity to network-wide shifts.

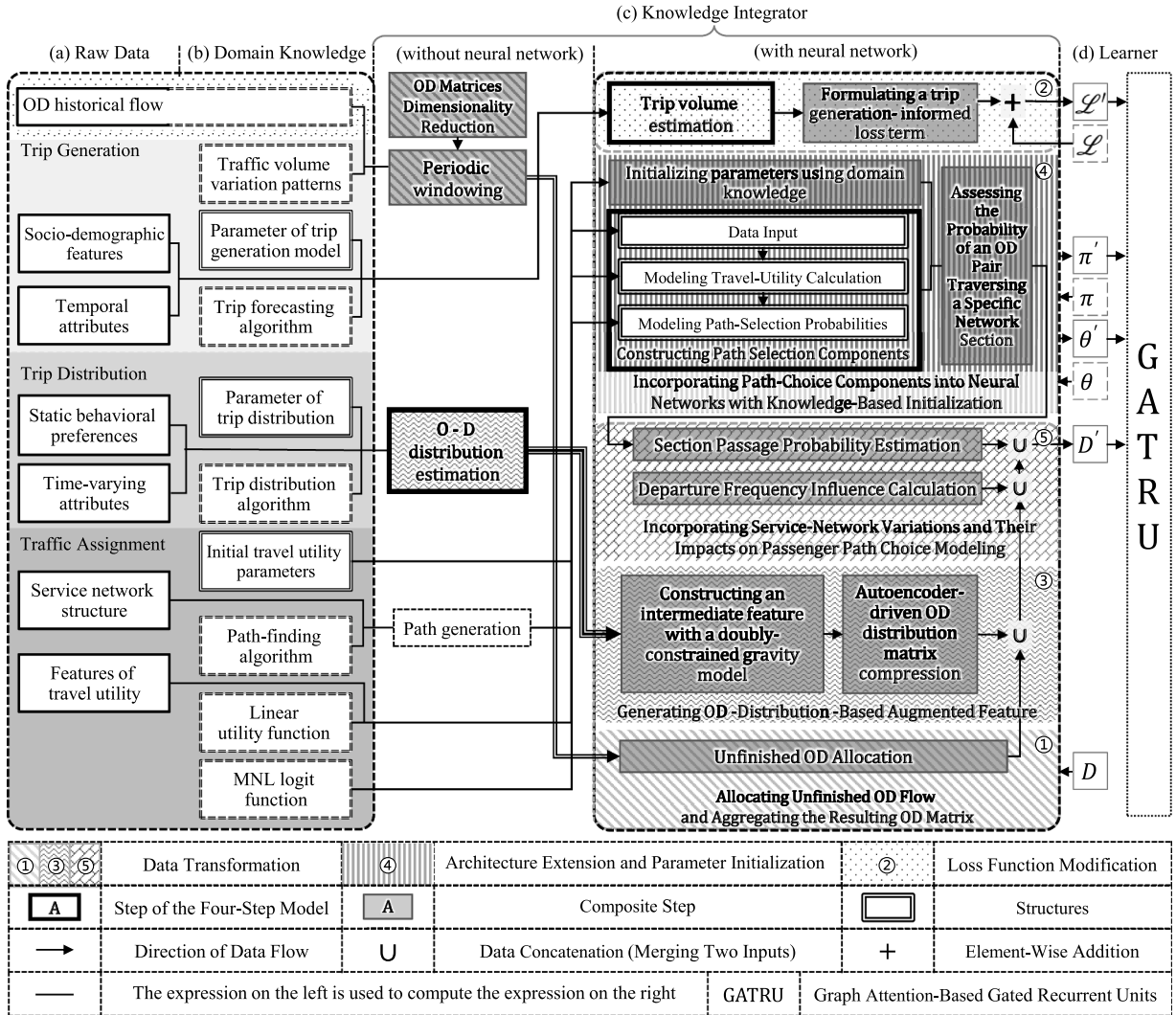


Fig. 2. Methodology for applying domain knowledge. This figure illustrates the transformation of transportation domain knowledge into deep learning components. Each Knowledge Integrator (marked ①–⑤) serves as a bridge between classical theory and data-driven optimization.

Influencing factors that alter path-level information affect how passengers choose their travel routes. To analyze their impact on the metro system, we use a multinomial logit (MNL) model. We calculate a utility score for each possible path. These scores then feed into a logit model, which determines the probability of each path being chosen. A Knowledge Integrator integrates both the utility layer and the logit model structure into a deep learning model, and initializes its parameters using domain knowledge. A Knowledge Integrator captures the path flow mechanism with a tripartite graph $G' = (OD \cup P \cup \Omega, F \cup U)$ linking OD pairs OD, paths P , and network sections Ω . Here, F is the set of edges connecting OD pairs to paths, and U connects paths to the network sections they traverse. Section-status matrices encode time-varying service-network states (e.g., service frequency and section availability). These matrices are appended as network-level augmentations to OD features and are also used to update the path-related service representations in the embedded assignment component.

In this way, we effectively analyze and integrate the impacts of these influencing factors on the metro system, enhancing the metro passenger-flow prediction model's robustness to variations.

3.2. The sequence-to-sequence prediction architecture

Our proposed framework, KNOWCAST, is built on a sequence-to-sequence (Seq2Seq) architecture, following previous works (Liu et al., 2023; Ke et al., 2021; Zhang et al., 2024). This architecture is shown in Fig. 3.

Let t_0 be the time step at which the prediction is made. The objective is to forecast OD flows for t_0 and subsequent time steps. The encoder processes the realized historical OD matrices and associated influencing factors over a lookback window of length l^h ,

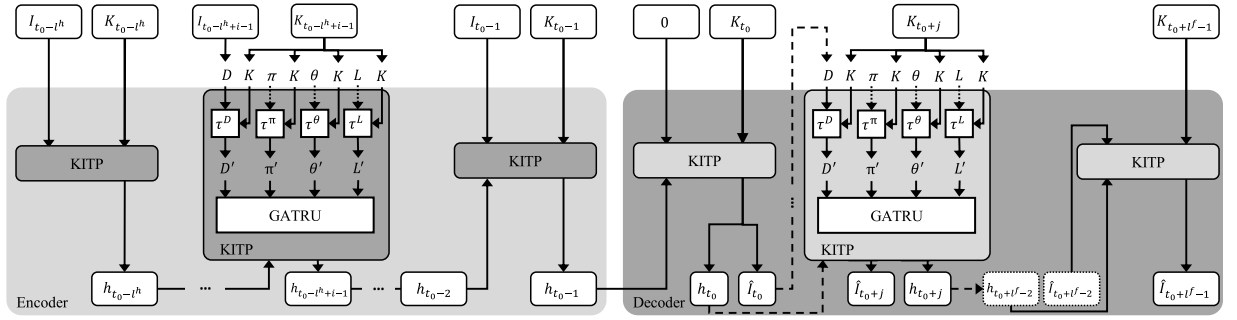


Fig. 3. The architecture of the KNOWCAST framework. It employs a Seq2Seq architecture with an encoder and decoder built from knowledge-integrated traffic predictor (KITEP) units.

namely from $t_0 - l^h$ to $t_0 - 1$. At each encoder step $i \in \{1, \dots, l^h\}$, a KITEP unit takes $\{I_{d,t_0-l^h+i-1}, K_{d,t_0-l^h+i-1}\}$ as input and updates the hidden state. The encoder hidden state is initialized to zero before the first encoder step. We index the encoder hidden state by the time of the most recently processed observed interval. Accordingly, after the KITEP unit processes the inputs at time t , the resulting hidden state is denoted by h_t . Hence, the encoder hidden states evolve as $h_{t_0-l^h}, \dots, h_{t_0-1}$, and the final encoder state h_{t_0-1} is passed to the decoder as the initial context state. Accordingly, the encoder uses only realized historical inputs from $t_0 - l^h$ to $t_0 - 1$, while t_0 is treated as the first prediction step of the forecast horizon.

The decoder then generates OD matrix predictions over a future forecast horizon of length l^f , namely from t_0 to $t_0 + l^f - 1$. At each decoder step, the KITEP unit receives the previous hidden state together with the previous OD input and the current domain-knowledge input, and outputs both the current OD prediction and the updated hidden state. At the first decoder step, the unit is initialized with the encoder state h_{t_0-1} and receives a zero-matrix input together with the current dynamic knowledge inputs $\{K_{d,t_0}^{\kappa}\}_{\kappa \in \{2,4,6,7\}}$, thereby producing the first prediction $\hat{I}_{d,t_0}$ and the updated hidden state h_{t_0} . From the second decoder step onward, the previous OD input differs between training and inference. During training, teacher forcing is applied: for $j = 1, \dots, l^f - 1$, the ground-truth OD matrix from the previous step, I_{d,t_0+j-1} , is fed as input together with the previous hidden state h_{t_0+j-1} and the current domain-knowledge input $\{K_{d,t_0+j}^{\kappa}\}_{\kappa \in \{2,4,6,7\}}$ to predict the matrix for the current step, $\hat{I}_{d,t_0+j}$, and to update the hidden state to h_{t_0+j} . During inference, where future ground-truth data is unavailable, for $j = 1, \dots, l^f - 1$, the decoder instead feeds back its own previous prediction, $\hat{I}_{d,t_0+j-1}$, as the OD input to the next KITEP step, together with h_{t_0+j-1} and $\{K_{d,t_0+j}^{\kappa}\}_{\kappa \in \{2,4,6,7\}}$, to generate the forecast for the current step, $\hat{I}_{d,t_0+j}$, and the updated hidden state h_{t_0+j} . Thus, the decoder hidden states evolve as $h_{t_0}, h_{t_0+1}, \dots, h_{t_0+l^f-1}$, while the predicted OD sequence is $\{\hat{I}_{d,t_0}, \dots, \hat{I}_{d,t_0+l^f-1}\}$. Notably, the KITEP decoder removes three Graph Attention Gated Recurrent Units (GATRU) that allocate destinations for ongoing, unfinished trips, as real-time unfinished-trip observations for future intervals are unavailable at the prediction origin. This structure allows the framework to learn complex temporal dependencies while ensuring that the encoding and decoding processes are guided by transportation domain knowledge and informed by influencing factors.

The problem statement of this research is as follows. Given the historical ridership data over the past l^h time intervals and the influencing factors available at the prediction origin, the goal is to train a model that learns a mapping function $f(\cdot)$ to predict the Actual OD matrices for the next l^f time intervals. The formulation is shown in Eq. (1):

$$\left[\left\{ \left\{ I_{d,t_0-l^h+i-1}^F, I_{d,t_0-l^h+i-1}^U \right\}_{i=1}^{l^h}, \{K^{\kappa}\}_{\kappa \in \{3,5\}}, \left\{ K_{d,t_0-l^h+i-1}^{\kappa} \mid i = 1, \dots, l^h + l^f, \kappa \in \{2,4,6,7\} \right\} \right\} \xrightarrow{f(\cdot)} \left\{ \hat{I}_{d,t_0+j-1}^A \right\}_{j=1}^{l^f} \quad (1)$$

In this formulation:

- $I_{d,t}^F$ denotes the finished-demand information at time t , while $I_{d,t}^U$ denotes the unfinished-demand information at time t . Their concrete processed representations used within the predictor are described in Section 4.2.
- $\hat{I}_{d,t}^A$ is the model's predicted Actual OD matrix at a future time t .
- $I_{d,t}^A$ is the corresponding ground-truth Actual OD matrix used during training to supervise $\hat{I}_{d,t}^A$.
- Static inputs K^{κ} are assumed not to change over the short-term forecast horizon. K^3 represents the static preference parameters for the trip distribution model (gravity model), and K^5 provides the domain-knowledge-based prior values used to initialize the route-choice utility coefficients. Dynamic inputs $K_{d,t}^{\kappa}$ exhibit temporal variability. $K_{d,t}^2$ includes explanatory variables for trip generation; $K_{d,t}^4$ includes dynamic variables for trip distribution; $K_{d,t}^6$ represents time-varying variables for the utility function; and $K_{d,t}^7$ describes the dynamic service network structure. The primary OD data are treated as the base inputs of the forecasting system and are therefore not indexed within the K^{κ} notation. Accordingly, the domain-knowledge-related inputs are indexed starting from $\kappa = 2$.

Here, the future-dated dynamic inputs $K_{d,t}^{\kappa}$ for $t \in \{t_0, \dots, t_0 + l^f - 1\}$ denote exogenous variables that are known in advance, operationally announced, or forecastable at the prediction origin. Accordingly, at forecasting time, KNOWCAST uses only information available at t_0 : AFC-derived historical OD inputs are available only up to $t_0 - 1$; scheduled service-network variables, such as timetables, planned headways, station closures, and line adjustments, are known in advance from official operation plans; and weather and

major-event indicators are obtained from short-term forecasts or publicly announced schedules. Therefore, the framework relies on information that is observable or forecastable at the decision time.

4. Key techniques of knowledge-integrated traffic predictor (KITP)

The Knowledge-Integrated Traffic Predictor (KITP) is the central component of the KNOWCAST framework. It incorporates domain knowledge into a network-level predictive model using the previously mentioned Knowledge Integrators. They are designed to analyze and integrate influencing factors by performing separate tasks such as enriching input features by capturing temporal regularities and modeling inter-station spatial correlations with a gravity model, introducing knowledge-informed loss terms, and utilizing dynamic network structures for knowledge-informed parameter initializations. The resulting knowledge-integrated features, loss functions, and initialized parameters are then leveraged by a Graph Attention Gated Recurrent Unit (GATRU) to jointly produce knowledge-aware network-level OD forecasting and path flow estimation.

4.1. Knowledge-driven modular design for enhanced flow forecasting and estimation

As illustrated in Fig. 4, the KITP's architecture is an implementation of the methodological framework introduced in Section 3.1 (see Fig. 2). Its horizontal layers directly correspond to the conceptual stages of (a) Raw Data; (b) Domain Knowledge (where Layer b.1 represents traffic-domain parameters, e.g., the initial coefficients for a path utility function; Layer b.2 describes the transportation behavioral rules, e.g., the functional form of a gravity model for trip distribution); (c) Knowledge Integrator; and (d) Learner.

Layer c is the action layer where domain knowledge from Layer b is applied to the data from Layer a. Through a series of transformations, the domain knowledge is applied to assess the impact on the metro system by performing linear transformations, arithmetic operations, and neural network transformations. Layer c in Fig. 4 is vertically organized into five Knowledge Integrators to perform separate tasks:

- (1) Allocating Unfinished OD Flow and Aggregating the Resulting OD Matrix: Component ① captures temporal regularities from historical OD flow data through periodic windowing and by addressing data sparsity via matrix aggregation.
- (2) Formulating a Trip Generation-Informed Loss Term: Component ② formulates a trip generation-informed loss term based on independently modeled station-level trip generation estimation, to guide the optimization process.
- (3) Generating OD-Distribution-Based Augmented Features: Component ③ generates an OD-Distribution-Based Adjustment Matrix to serve as an augmented feature for the primary OD matrix, thereby informing the model about how inter-station travel patterns are likely to shift due to these influences.
- (4) Incorporating Path-Choice Components into Neural Networks with Knowledge-Based Initialization: Component ④ functions as a differentiable assignment layer. It leverages domain knowledge of traffic assignment to both incorporate a logit-model-based component into the deep learning framework and initialize a set of trainable parameters with knowledge-informed values. This design allows the model to infer path flows directly from OD forecasts without requiring path-level ground truth, ensuring that the estimated flows remain behaviorally consistent with the predicted OD demand.
- (5) Incorporating Service-Network Variations and Their Impacts on Passenger Path Choice Modeling: Component ⑤ augments the OD features using compressed feature representations of path-level choices and service frequency. These representations are generated from network-level service information.

Taken together, the predictor-side OD feature representation consists of the primary historical OD-flow input and three knowledge-integrated augmentations: the OD-distribution-based feature from Component ③, the compressed path-choice representation from Component ④/⑤, and the compressed service-frequency representation from Component ⑤. Thus, the historical OD input provides the demand baseline, while the three augmented components respectively encode distributional shifts, path-choice information, and time-varying service-network conditions.

Beyond feature extraction, these Knowledge Integrators serve a critical function by acting as structural regularizers. While traditional deep learning often requires explicit regularization such as L_1 or L_2 norms to prevent overfitting, the integration of theoretical models in KNOWCAST imposes strong inductive biases that constrain the neural network's hypothesis space. For instance, the Logit-based path choice component (Component ④) and the Gravity-based distribution logic (Component ③) force the model to seek solutions adhering to transportation principles rather than memorizing high-frequency noise or idiosyncratic patterns in the training data. This design is intended to improve robustness and generalizability. For detailed explanation of each Knowledge Integrator, see Sections 4.2–4.6:

4.2. Allocating unfinished OD flow and aggregating the resulting OD matrix (component ①)

This Knowledge Integrator first addresses data sparsity by transforming the raw OD matrices into the compressed representation used by the predictor. This adjustment preserves the dominant OD flows while aggregating low-volume destinations into a residual category.

It then generates a more complete real-time snapshot of network-wide travel demand by allocating unfinished trips within the same compressed destination space. This is achieved in two stages. First, a periodic windowing procedure organizes recurrent historical destination patterns into historical reference inputs. Second, the unfinished OD allocation module combines these historical destination patterns with the current unfinished-trip counts to construct the auxiliary inputs used by the predictor.

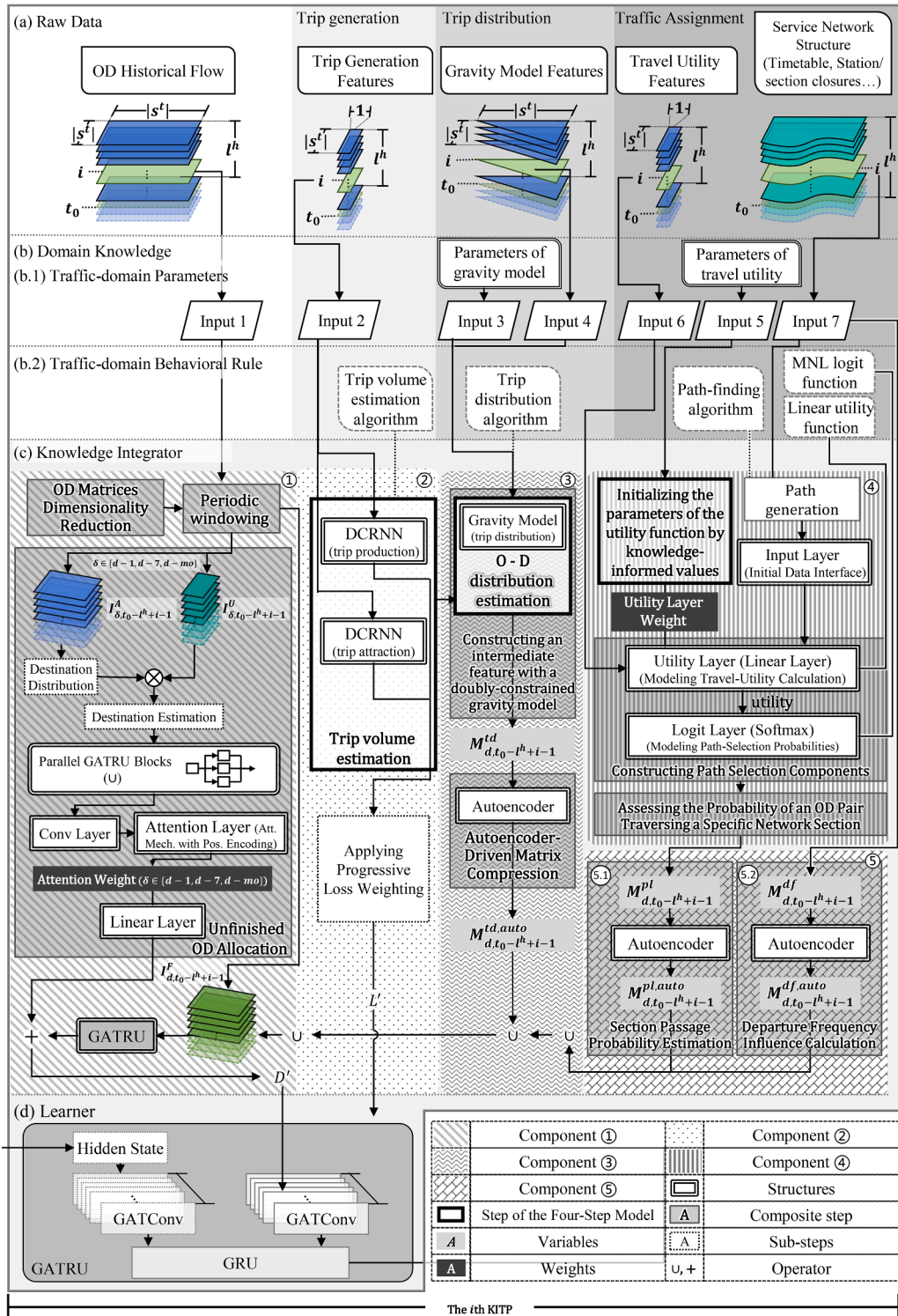


Fig. 4. KITP: The core component of the KNOWCAST framework's encoder and decoder. This diagram represents the technical implementation of the methodology in Fig. 2 (Horizontal View), where each Knowledge Integrator corresponds strictly to the logical layers defined previously.

4.2.1. Input 1: historical origin-destination (OD) flow data

The model's primary input is the historical OD flow data derived directly from the metro's AFC data, as defined in Eq. (1). Each raw AFC record contains a unique card ID, an origin station (tap-in), an entry time, a destination station (tap-out), and an exit time. This provides a direct, observed record of passenger OD trips. To structure this data for short-term forecasting, we define a reference time t_0 , which is the precise moment a prediction is made. All historical data from past intervals t (where $t < t_0$) is processed relative to this reference time t_0 . This processing yields three distinct data structures for any given historical time interval (d, t) :

Let the set of all metro stations be STN, where $|\text{STN}|$ is the total number of stations. Let $\mathcal{D}$ denote the set of service days in the raw dataset and let $\mathcal{T} = \{1, \dots, \rho\}$ denote the set of time intervals within each day, where ρ is the number of intervals per day. For any given day-time interval pair $(d, t) \in \mathcal{D} \times \mathcal{T}$, the raw AFC records are processed from the dataset $I_{\text{raw}} = \{I_{d,t}^{\text{raw}} \mid d \in \mathcal{D}, t \in \mathcal{T}\}$ into the following structured matrices and vectors:

- Actual OD Matrix ($I_{d,t}^{\text{raw},A} \in \mathbb{R}^{|\text{STN}| \times |\text{STN}|}$): Serving as the ground-truth travel demand for interval t and our prediction target. Each element $I_{d,t}^{\text{raw},A}(m, n)$ represents the total number of passengers who began their journey (tapped-in) at origin m during interval t and eventually completed it (tapped-out) at destination n , based on the complete AFC transaction history.
- Finished OD Matrix ($I_{d,t}^{\text{raw},F} \in \mathbb{R}^{|\text{STN}| \times |\text{STN}|}$): Each element $I_{d,t}^{\text{raw},F}(m, n)$ tracks the number of passengers who started at station m during interval t and completed their trip at station n at or before the reference time t_0 . This matrix represents the OD flow from interval t that is already known and completed by the time of prediction.
- Unfinished Trip Vector ($I_{d,t}^{\text{raw},U} \in \mathbb{R}^{|\text{STN}| \times 1}$): Each element $I_{d,t}^{\text{raw},U}(m)$ quantifies the number of passengers who entered (tapped-in) at station m during interval t but have not yet exited (tapped-out) the system by the reference time t_0 . These passengers are still in-transit, and their final destinations are unknown at t_0 .

For each origin station, the total actual departures must equal the sum of completed and ongoing trips by forecast time t_0 . The theoretical relationship between the matrices $I_{d,t}^{\text{raw},A}$, $I_{d,t}^{\text{raw},U}$, and $I_{d,t}^{\text{raw},F}$ is given in Eq. (2),

$$\sum_{n \in \text{STN}} I_{d,t}^{\text{raw},A}(m, n) = \sum_{n \in \text{STN}} I_{d,t}^{\text{raw},F}(m, n) + I_{d,t}^{\text{raw},U}(m), \quad \forall m \in \text{STN} \quad (2)$$

4.2.2. OD matrices dimensionality reduction via selective aggregation

To address the high dimensionality and sparsity that typically characterize OD matrices in large-scale metro systems, we convert each raw OD matrix into a compressed representation by retaining the dominant destination flows for each origin and aggregating the remaining low-volume destinations into a residual column, following the general idea of Liu et al. (2020).

The compression is performed on an origin-by-origin basis. For each origin station m , destination stations are ranked according to their cumulative flow over the training period. We then retain the top $|\text{RDS}|$ destinations explicitly and aggregate all remaining destinations into one additional column. Accordingly, for each origin m , we define an ordered retained destination set

$$\text{RDS}_m = \{n_m^{(1)}, n_m^{(2)}, \dots, n_m^{(|\text{RDS}|)}\},$$

where $|\text{RDS}|$ is the number of explicitly retained destinations for each origin. Here, the subscript m indicates that the retained destination set is defined separately for each origin row, and $n_m^{(\tilde{n})}$ denotes the actual destination station occupying the $\tilde{n}$ th retained slot for origin m . In implementation, the mapping RDS_m is fixed once before training using the training-period ranking criterion and is then kept unchanged for all time steps in training, validation, and testing. By construction, each retained set RDS_m has the same cardinality, i.e., $|\text{RDS}_m| = |\text{RDS}|$ for all origins m .

The transformation from the raw matrix $I_{d,t}^{\text{raw},X}$ to the compressed matrix $I_{d,t}^X$ (where $X \in \{A, F\}$) is therefore defined in a fixed compressed destination space. The compressed matrix has dimension $|\text{STN}| \times (|\text{RDS}| + 1)$: the first $|\text{RDS}|$ columns correspond to the retained destinations, and the final column stores the aggregated residual flow to all non-retained destinations.

For the retained destinations, the compressed entries are defined by Eq. (3):

$$I_{d,t}^X(m, \tilde{n}) = I_{d,t}^{\text{raw},X}(m, n_m^{(\tilde{n})}), \quad \tilde{n} = 1, 2, \dots, |\text{RDS}| \quad (3)$$

The residual aggregated column is defined by Eq. (4):

$$I_{d,t}^X(m, |\text{RDS}| + 1) = \sum_{n \in \text{STN} \setminus \text{RDS}_m} I_{d,t}^{\text{raw},X}(m, n) \quad (4)$$

Applying this row-wise procedure for all origin stations produces the final compressed matrix $I_{d,t}^X$. This representation preserves the dominant ridership structure while reducing dimensionality and mitigating the influence of highly sparse tail destinations. It also preserves the total outflow of each origin, since the retained flows and the residual aggregated column together account for all destinations in the corresponding raw OD row.

4.2.3. Periodic windowing for historical data structuring

The periodic windowing approach is founded on the observation that traffic dynamics exhibit strong daily, weekly, and monthly recurrences. To capture these cyclical patterns, periodic windowing is performed in the same compressed OD space defined above. Its purpose is to organize historical destination patterns into reference inputs for the subsequent unfinished-trip allocation stage.

A periodic windowing operator $\mathcal{W}$ maps each time point (d, t) to an enriched set of compressed historical snapshots. Let $\mathcal{I}_{d-\delta, t}$ denote one compressed historical snapshot at time interval t on day $d - \delta$, and let $\mathcal{I}_{d, t}$ denote the corresponding enriched collection. The periodic windowing operator $\mathcal{W}$ is defined in Eq. (5):

$$\mathcal{W} : (d, t, \mathbf{I}) \mapsto \mathcal{I}_{d, t} = \{\mathcal{I}_{d-\delta, t} \mid \delta \in \{0, 1, 7, \text{mo}\}\} \quad (5)$$

where δ is a periodic offset from the set $\{0, 1, 7, \text{mo}\}$, corresponding respectively to the current day, the previous day, the same weekday in the previous week, and the same calendar day in the previous month.

For a prediction at target time t_0 on day d_0 with a look-back window of l^h intervals, the input sequence is formed by arranging the windowed outputs for successive historical time steps in chronological order, as shown in Eq. (6).

$$\mathcal{I}_{d_0, [t_0-l^h, t_0-1]} = (\mathcal{W}(d_0, t_0-l^h, \mathbf{I}), \mathcal{W}(d_0, t_0-l^h+1, \mathbf{I}), \dots, \mathcal{W}(d_0, t_0-1, \mathbf{I})) \quad (6)$$

4.2.4. Unfinished OD allocation

To provide a more complete real-time snapshot of OD flow volume, this component takes the periodically structured historical inputs from Section 4.2.3 and allocates unfinished trips in the same compressed destination space used by the predictor. Although the general periodic windowing operator in Section 4.2.3 is defined over $\delta \in \{0, 1, 7, \text{mo}\}$, the unfinished-trip allocation stage uses only $\delta \in \{1, 7, \text{mo}\}$. The same-day case $\delta = 0$ is excluded here because the current-day actual destination distribution is not fully observable at prediction time t_0 .

For each historical reference source indexed by δ , this component constructs a destination probability distribution $D_{d_0-\delta, t_0-l^h+i-1}(m, \tilde{n})$ for unfinished trips originating from station m , where $\tilde{n}$ indexes the compressed destination columns. These distributions are derived from the historical compressed OD matrices under different periodic windows, as shown in Eq. (7).

$$D_{d_0-\delta, t_0-l^h+i-1}(m, \tilde{n}) = \begin{cases} \frac{I_{d_0-\delta, t_0-l^h+i-1}^A(m, \tilde{n})}{\sum_{\tilde{n}'=1}^{|\text{RDS}|+1} I_{d_0-\delta, t_0-l^h+i-1}^A(m, \tilde{n}')}, & \text{if } \sum_{\tilde{n}'=1}^{|\text{RDS}|+1} I_{d_0-\delta, t_0-l^h+i-1}^A(m, \tilde{n}') > 0, \\ \frac{1}{|\text{RDS}|+1}, & \text{otherwise,} \end{cases} \quad (7)$$

$\tilde{n} = 1, \dots, |\text{RDS}|+1, \quad \delta \in \{1, 7, \text{mo}\}.$

If the historical row total for origin m is zero under a given reference source δ , a uniform distribution over the compressed destination slots is used as a fallback to avoid division by zero.

This historical destination distribution is then applied to the current unfinished-trip counts to produce three current-day unfinished-trip allocation matrices, one for each historical reference source $\delta \in \{1, 7, \text{mo}\}$. The calculation is performed by scaling the compressed historical destination probability distribution for each origin station by the corresponding current unfinished-trip count. This row-wise allocation is formally defined for each origin-compressed-destination pair $(m, \tilde{n})$ in Eq. (8):

$$\tilde{I}_{d_0, t_0-l^h+i-1}^{U, \delta}(m, \tilde{n}) = D_{d_0-\delta, t_0-l^h+i-1}(m, \tilde{n}) \cdot I_{d_0, t_0-l^h+i-1}^{\text{raw}, U}(m), \quad \tilde{n} = 1, \dots, |\text{RDS}|+1, \quad \delta \in \{1, 7, \text{mo}\}. \quad (8)$$

Here, $\tilde{I}_{d_0, t_0-l^h+i-1}^{U, \delta}$ denotes the current-day unfinished-trip allocation matrix constructed using the historical destination distribution associated with reference source δ .

Each unfinished-trip allocation sequence is processed sequentially by a corresponding Graph Attention Gated Recurrent Unit (GATRU) layer over the historical window $i \in \{1, \dots, l^h\}$ to generate temporally updated hidden states $\mathbf{H}_{m, i}^\delta$ that incorporate spatiotemporal features, as shown in Eq. (9):

$$\mathbf{H}_i^\delta = \text{GATRU}(\tilde{I}_{d_0, t_0-l^h+i-1}^{U, \delta}, \mathbf{H}_{i-1}^\delta), \quad \delta \in \{1, 7, \text{mo}\}. \quad (9)$$

Here, $\mathbf{H}_i^\delta$ denotes the collection of hidden states for all nodes at step i under historical reference source δ , and $\mathbf{H}_{m, i}^\delta$ denotes the hidden state of node m extracted from $\mathbf{H}_i^\delta$.

After obtaining the final hidden states at step l^h (i.e., $\mathbf{H}_{m, l^h}^1$, $\mathbf{H}_{m, l^h}^7$, and $\mathbf{H}_{m, l^h}^{\text{mo}}$), we concatenate them into $\mathbf{H}_m^{\text{combined}}$. Positional encoding is then incorporated into the fused representation before the attention-based gating operation, as shown in Eq. (10).

$$\mathbf{W}_m^{\text{attn}} = \text{GatingLayer}(\mathbf{Z}_m^{\text{combined}}, \mathbf{Z}_m^{\text{combined}}, \mathbf{Z}_m^{\text{combined}}), \quad \mathbf{Z}_m^{\text{combined}} = \mathbf{H}_m^{\text{combined}} + \mathbf{P}^{\text{pos}}. \quad (10)$$

Here, $\mathbf{P}^{\text{pos}}$ denotes the positional encoding incorporated into the fused hidden representation before the attention-based gating operation.

This attention-based gating layer produces gating weights $\mathbf{W}_m^{\text{attn}}$, which are then split into $\mathbf{W}_m^1$, $\mathbf{W}_m^7$, and $\mathbf{W}_m^{\text{mo}}$, corresponding to $\mathbf{H}_{m, l^h}^1$, $\mathbf{H}_{m, l^h}^7$, and $\mathbf{H}_{m, l^h}^{\text{mo}}$, respectively, as defined in Eq. (11).

$$[\mathbf{W}_m^1, \mathbf{W}_m^7, \mathbf{W}_m^{\text{mo}}] = \mathbf{W}_m^{\text{attn}} \quad (11)$$

The final unfinished-trip hidden state, $\mathbf{H}_m^{\text{final}}$, is formed by applying a gating mechanism that weights the contribution of each historical hidden state. The attention weights $\mathbf{W}_m^\delta$ for each period δ are passed through a sigmoid activation function, and the resulting gates are used to scale their corresponding hidden states $\mathbf{H}_m^\delta$. These weighted states are then summed to produce the final state, as shown in Eq. (12).

$$\mathbf{H}_m^{\text{final}} = \sum_{\delta \in \{1,7,\text{mo}\}} \sigma(\mathbf{W}_m^\delta) \cdot \mathbf{H}_m^\delta \quad (12)$$

where $\sigma(\cdot)$ denotes the element-wise sigmoid activation function.

Concurrently, the compressed Finished OD Matrix $\mathbf{I}_{d,t}^F$ is processed by a parallel GATRU layer to encode its own spatiotemporal features, yielding the hidden state $\mathbf{H}_m^{\text{finished}}$.

The final hidden state $\mathbf{H}_m^{\text{final}}$ is combined with $\mathbf{H}_m^{\text{finished}}$ to form the encoder output for node m , as shown in Eq. (13).

$$\mathbf{O}^{(m)} = \mathbf{H}_m^{\text{final}} + \mathbf{H}_m^{\text{finished}} \quad (13)$$

4.3. Formulating a trip generation-informed loss term (component ②)

To improve training efficiency by encouraging the model to generate OD flows that align with aggregate station-level trip production and attraction estimates, we introduce a domain-knowledge-integrated penalty term into the loss function. This penalty measures the divergence between the predicted OD flows and independently estimated station-level trip generation volumes. By incorporating this loss term, the model is encouraged to align its predictions with domain-knowledge-informed expectations, particularly during the early stages of training.

To further improve training efficiency, we adopt a Progressive Loss Weighting strategy. This approach gradually adjusts the relative influence of different loss components during training. It allows the model to first focus on the more straightforward task of matching trip generation volumes before progressively shifting attention to the more complex objective of accurate OD flow prediction. This learning process facilitates better convergence and performance.

4.3.1. Input 2: features of trip volume estimation

Input 2, denoted $K_{d,t_0 \rightarrow t_0+h+1}^2$, contains the explanatory variables used to inform Component ②. These variables capture both estimated long-term socio-demographic characteristics associated with the station and observed short-run temporal effects influencing passenger generation/attraction intensity at each time step t .

The explanatory factors comprise socio-demographic and temporal attributes (Huang et al., 2010). The socio-demographic variables are (i) the local population activity level, represented by an estimate of the catchment population size, PopAct_m ; (ii) the dominant age structure, often represented by the proportion of the working-age population, $\text{AgePct}_m^{\text{work}}$; (iii) the local education profile, defined as the population's percentage distribution across four educational attainment levels: primary (primary or below), junior middle (junMid), senior high or technical secondary (senHigh), and junior college or above (tertiary). This profile is denoted by $\text{EduPct}_{m,\text{EDU}}$, where $\text{EDU} \in \{\text{primary}, \text{junMid}, \text{senHigh}, \text{tertiary}\}$; and (iv) the average household size $\overline{\text{HHSIZE}}_m$.

The temporal dimension includes: (i) seasonality indicators, $\text{Season}_d \in \{\text{Spring}, \text{Summer}, \text{Autumn}, \text{Winter}\}$; (ii) a weekpart indicator, $\text{Wknd}_d = 1$ for weekends and public holidays and 0 otherwise; (iii) a continuous time-of-day variable, $\text{TOD}_t \in [0, 24) \text{h}$, expressed as the number of hours elapsed since midnight; (iv) an event-driven impact score, $\text{Event}_{m,d,t}$, defined as an intensity index between 0 and 1 for major concerts, exhibitions, or sports games occurring within 1 km of station m at time (d, t) , calculated as a weighted sum of expected attendance and temporal proximity to the event's start time.

For the pre-trained model within Component ②, these covariates are assembled into a feature vector $\mathbf{k}_{m,d,t}^{TG}$, as shown in Eq. (14):

$$\mathbf{k}_{m,d,t}^{TG} = \underbrace{[\text{PopAct}_m, \text{AgePct}_m^{\text{work}}, \text{EduPct}_{m,\text{EDU}}, \overline{\text{HHSIZE}}_m]}_{\text{Socio-Demographic}} \underbrace{[\text{Wknd}_d, \text{TOD}_t, \text{Season}_d, \text{Event}_{m,d,t}]}_{\text{Temporal}} \quad (14)$$

The specific features used and their generation for the study area are detailed in Section 5.1.2.

4.3.2. Employing DCRNN to capture the spatiotemporal dependencies in trip generation

To ensure that localized disruptions in passenger flow are effectively captured in OD flow predictions, we incorporate station-level dynamics into the OD forecasting process. We independently forecast the trip generation and attraction at each station and use the divergence between these estimates and the predicted OD flows as domain-knowledge-integrated loss terms. These terms are integrated into the loss function to guide the training process, thereby improving the model's sensitivity to deviations from regular cyclical patterns.

We begin by representing the spatial structure for trip generation as a weighted directed graph $G_{\text{Taz}} = (V_{\text{Taz}}, E_{\text{Taz}}, W_{\text{Taz}})$. V_{Taz} denotes a set of vertices, where each vertex $v \in V_{\text{Taz}}$ corresponds to a Traffic Analysis Zone (TAZ) centered around a metro station. A TAZ serves as a spatial unit that aggregates nearby nodes and links with similar travel behavior or strong traffic correlation. E_{Taz} denotes the physical metro links connecting these zones, and W_{Taz} represents the weighted adjacency matrix governing the spatial diffusion process in the DCRNN. The model is driven by features of each station's TAZ, which are used to model trip attraction and trip production.

To forecast future trip generation and attraction, we employ a Diffusion Convolutional Recurrent Neural Network (DCRNN) within a Sequence-to-Sequence (Seq2Seq) architecture for prediction. This model captures spatiotemporal dependencies by first using Diffusion Convolutional layers to model the spatial influence between neighboring stations through a graph-based information diffusion process. The resulting spatially aware features are then fed into Gated Recurrent Units (GRUs), which model the temporal dynamics of the traffic flow over time.

To enforce consistency between the trip-generation module and the OD forecast, we introduce a trip-generation-informed loss term $\mathcal{L}_g$. Here, $\hat{\mathbf{I}}_{d,t}^A$ denotes the predictor output, which is defined in the row-wise compressed destination space introduced in Section 4.2.2. The production-side term is computed directly from the row totals of $\hat{\mathbf{I}}_{d,t}^A$, since the retained destination entries and the residual aggregated column together preserve the total predicted departures from each origin.

For the attraction-side term, $\text{DestTotal}_n(\hat{\mathbf{I}}_{d,t}^A)$ combines the retained entries that explicitly correspond to destination n with a fixed training-period allocation of the residual aggregated column. Specifically, the sums over (d', t') in the residual-allocation fraction below are computed over the training period, as shown in Eq. (15).

$$\begin{aligned} \text{DestTotal}_n(\hat{\mathbf{I}}_{d,t}^A) = & \sum_{\substack{m \in \text{STN}, n \in \text{RDS}_m \\ \tilde{n} \in \{1, \dots, |\text{RDS}|\} : n_m^{(\tilde{n})} = n}} \hat{\mathbf{I}}_{d,t}^A(m, \tilde{n}) \\ & + \sum_{\substack{m \in \text{STN} \\ n \notin \text{RDS}_m}} \left(\frac{\sum_{d', t'} \mathbf{I}_{d', t'}^{\text{raw}, A}(m, n)}{\sum_{d', t'} \sum_{n' \in \text{STN} \setminus \text{RDS}_m} \mathbf{I}_{d', t'}^{\text{raw}, A}(m, n')} \right) \hat{\mathbf{I}}_{d,t}^A(m, |\text{RDS}| + 1). \end{aligned} \quad (15)$$

If the denominator of the residual-allocation fraction is zero, the corresponding fraction is replaced by a uniform share $1/(|\text{STN}| - |\text{RDS}|)$ over the non-retained destinations for that origin. In practical terms, this operator uses the training-period destination shares within each origin-specific residual group to approximately allocate the predicted residual-column flow back to original destinations. This definition approximates destination-level arrivals from the compressed prediction output, without treating the residual aggregated column as a directly recoverable single destination. The resulting loss is given in Eq. (16).

$$\mathcal{L}_g(P, A, \hat{\mathbf{I}}^A) = \sum_{d, t} \left[\sum_m \left(P_{m, d, t} - \sum_{\tilde{n}=1}^{|\text{RDS}|+1} \hat{\mathbf{I}}_{d, t}^A(m, \tilde{n}) \right)^2 + \sum_n \left(A_{n, d, t} - \text{DestTotal}_n(\hat{\mathbf{I}}_{d, t}^A) \right)^2 \right] \quad (16)$$

where, for a given day d and time interval t , $P_{m, d, t}$ is the estimated total number of trips produced by origin zone m , $A_{n, d, t}$ is the estimated total number of trips attracted to destination zone n , and $\hat{\mathbf{I}}_{d, t}^A$ denotes the predicted OD matrix output by the predictor. Accordingly, $\sum_{\tilde{n}=1}^{|\text{RDS}|+1} \hat{\mathbf{I}}_{d, t}^A(m, \tilde{n})$ gives the predicted departures from origin m in the compressed destination space, whereas $\text{DestTotal}_n(\hat{\mathbf{I}}_{d, t}^A)$ gives the approximate predicted arrivals associated with destination n under the fixed residual-allocation rule used in the implementation.

4.3.3. Applying progressive loss weighting

To leverage the trip generation domain knowledge to guide the model's initial learning without compromising the final performance on the OD prediction task, we adopt a Progressive Loss Weighting strategy (Chen et al., 2018; Guo et al., 2018; Heydari et al., 2019; Liu et al., 2019b). This approach addresses the challenges of training instability that often arise when directly optimizing the highly complex OD prediction task. It enables the model's attention to gradually transition from the simpler trip generation task to the more complex OD prediction task.

Specifically, during the early stages of training, we assign a higher weight to the trip generation-informed loss term. As training progresses, the weight of the trip generation-informed loss term is linearly decayed, shifting the optimization focus toward the core OD prediction task. This strategy allows the model to fully leverage the domain knowledge from the trip generation phase while avoiding being overwhelmed by the optimization complexity of OD prediction in the early stages, thereby promoting more stable and efficient convergence.

We adopt a linear decay strategy for progressive loss weighting. Specifically, the weight $w_g(\epsilon)$ of the trip-generation-informed loss decreases linearly with the training epoch, as shown in Eq. (17).

$$w_g(\epsilon) = w_g(0) \cdot \max(1 - \alpha_{\text{decay}} \cdot \epsilon, 0) \quad (17)$$

where $w_g(0)$ is the initial weight of the trip-generation-informed loss, α_{decay} is the relative linear decay rate per epoch, and ϵ is the current training epoch.

The primary forecasting objective is the OD reconstruction loss, denoted by $\mathcal{L}_{\text{OD}}$. In implementation, the model outputs and the corresponding ground-truth OD matrices are first transformed back to the original flow-value scale, and the loss is then computed as the mean absolute error over all prediction targets in the training set. Formally, the OD forecasting loss is defined as

$$\mathcal{L}_{\text{OD}} = \frac{1}{N_{\text{train}} |\text{STN}| (|\text{RDS}| + 1)} \sum_{d, t} \sum_{m \in \text{STN}} \sum_{\tilde{n}=1}^{|\text{RDS}|+1} \left| \hat{\mathbf{I}}_{d, t}^A(m, \tilde{n}) - \mathbf{I}_{d, t}^A(m, \tilde{n}) \right|, \quad (18)$$

where N_{train} denotes the total number of predicted OD matrices included in the training loss.

Ultimately, the Progressive Loss Weighting strategy culminates in a unified objective function. The total loss $\mathcal{L}_{\text{total}}^{(\epsilon)}$ at training epoch ϵ is computed as the sum of the primary OD forecasting loss $\mathcal{L}_{\text{OD}}$ in Eq. (18) and the dynamically weighted trip-generation-informed term $\mathcal{L}_g$, as shown in Eq. (19):

$$\mathcal{L}_{\text{total}}^{(\epsilon)} = \mathcal{L}_{\text{OD}} + w_g(\epsilon) \cdot \mathcal{L}_g \quad (19)$$

4.4. Generating OD-distribution-based augmented feature (component ③)

To capture shifts in inter-station travel patterns, we introduce a knowledge-informed feature to the OD matrix. We first estimate the trip distribution using a gravity model and then compress the resulting distribution matrix through an Autoencoder. The compressed representation, referred to as the OD-Distribution-Based Adjustment Matrix, is used as an augmented feature alongside the original OD matrix to enhance the model's awareness of distributional dynamics.

4.4.1. Input 3 and 4: preference parameters and features of trip distribution

Input 3 (K^3) represents the pre-calibrated static parameter(s) of the deterrence function in Eq. (20), such as the deterrence coefficient ζ . These parameters reflect long-term behavioral preferences and are treated as time-invariant over the short-term forecasting horizon.

Input 4 ($K_{d,t_0-l^h+i-1}^4$) represents the time-varying inputs in Eq. (20), specifically the dynamic generalized travel impedance $c_{mn,d,t}$ and the temporally varying trip production and attraction terms $P_{m,d,t}$ and $A_{n,d,t}$. These variables vary over time as service conditions, demand patterns, and exogenous operational factors change. In particular, the generalized impedance $c_{mn,d,t}$ may combine actual travel time, fare-related monetary cost, and perceived discomfort (e.g., due to crowding or unreliable service) between an origin m and a destination n . In implementation, the production and attraction vectors are balanced by construction for each (d, t) , i.e., $\sum_m P_{m,d,t} = \sum_n A_{n,d,t}$.

4.4.2. Constructing an intermediate feature with a doubly-constrained gravity model

To capture dynamic shifts in inter-station travel patterns, particularly those arising from systemic demand re-distributions that the gravity model is well-suited to capture, we construct an enhanced feature using a doubly-constrained gravity model. The trip-distribution estimates are updated dynamically at each time step to reflect current network conditions and event-driven surges. The resulting enhanced features (i.e., the OD-Distribution-Based Matrix) are finally embedded into the feature set provided to the neural network. It is important to note that the deterrence parameter of this gravity model (ζ) is pre-calibrated offline using the historical data and is held static during the main model's training. Meanwhile, the balancing coefficients ($BO_{m,d,t}$ and $BD_{n,d,t}$) are calculated dynamically at each time step via the Iterative Proportional Fitting (IPF) procedure. This is a deliberate design choice to provide a stable, transportation domain knowledge-grounded feature based on long-term patterns. The temporal dynamics for this component are captured by its time-varying inputs ($P_{m,d,t}$ and $A_{n,d,t}$) and the subsequent fine-tuning of its Autoencoder (Section 4.4.3).

We apply the doubly-constrained gravity model (Williams, 1976) to estimate trip distributions, using the results from the previous stage of trip generation. This doubly-constrained gravity model prescribes that the expected flow $\gamma_{(m,n),d,t}^{\text{TD}}$ between an origin location m and a destination location n on day d at time t is generated according to Eq. (20):

$$\gamma_{(m,n),d,t}^{\text{TD}} = BO_{m,d,t} P_{m,d,t} BD_{n,d,t} A_{n,d,t} f(c_{mn,d,t}) \quad (20)$$

where the deterrence function is specified as an exponential decay function, $f(c) = e^{-\zeta c}$ with $\zeta > 0$; $P_{m,d,t}$ and $A_{n,d,t}$ represent the trip production and attraction volumes for station m and n on day d at time t , respectively; the two proportionality constants are: $BO_{m,d,t} = [\sum_n BD_{n,d,t} A_{n,d,t} f(c_{mn,d,t})]^{-1}$, $BD_{n,d,t} = [\sum_m BO_{m,d,t} P_{m,d,t} f(c_{mn,d,t})]^{-1}$. These constants are calibrated through an iterative proportional fitting procedure.

4.4.3. Autoencoder-driven matrix compression

To reduce the dimensionality of the destination distribution matrix while preserving essential spatial information, we apply an Autoencoder-based compression technique. The original matrix $M_{d,t}^{\text{TD}}$ (where $\gamma_{(m,n),d,t}^{\text{TD}}$ denotes the (m, n) th element, representing the attraction levels between stations) is first compressed using the matrix compression method introduced in Section 4.2.2, and then fed into an Autoencoder. The Autoencoder learns a low-dimensional latent representation $M_{d,t}^{\text{TD,auto}}$ that captures the most salient non-linear patterns in the data.

This compression is handled in two phases.

First, the Autoencoder is pre-trained independently to learn a low-dimensional latent representation. This Autoencoder consists of an encoder f_{encoder} (with parameters θ_{enc}) and a decoder f_{decoder} (with parameters θ_{dec}). The objective is to learn a latent representation $\hat{M} = f_{\text{encoder}}(M; \theta_{\text{enc}})$ that captures salient, general non-linear patterns by minimizing the reconstruction loss $\mathcal{L}_{\text{recon}}$ (Mean Squared Error):

$$(\theta_{\text{enc}}^*, \theta_{\text{dec}}^*) = \arg \min_{\theta_{\text{enc}}, \theta_{\text{dec}}} \mathcal{L}_{\text{recon}}(M, f_{\text{decoder}}(\hat{M}; \theta_{\text{dec}}))$$

Second, the pre-trained encoder f_{encoder} (initialized with θ_{enc}^*) is integrated into the main KNOWCAST framework and is fine-tuned jointly with the predictive model's parameters θ_G . During this joint training, the encoder's parameters θ_{enc} are updated via

backpropagation from the final prediction loss, $\mathcal{L}_{\text{pred}}$. This process shifts the encoder's objective:

$$(\theta_G^*, \theta_{\text{enc}}^{**}) = \arg \min_{\theta_G, \theta_{\text{enc}}} \mathcal{L}_{\text{pred}}(Y_{\text{true}}, G_{\text{KNOWCAST}}(X, f_{\text{encoder}}(M; \theta_{\text{enc}}); \theta_G))$$

Here, X represents the other inputs to the main model G_{KNOWCAST} , and Y_{true} is the ground-truth prediction target. This fine-tuning stage addresses the concern of information loss. While compression is inherently lossy, the joint training optimizes the encoder to selectively preserve features that are most relevant to the downstream forecasting task, while learning to discard information that is irrelevant or noisy for predictive accuracy.

The resulting low-dimensional embedding $\hat{M} = f_{\text{encoder}}(M; \theta_{\text{enc}}^{**})$ serves as an enhanced, task-specific feature input for the predictive model, enhancing its ability to capture inter-station dynamics and spatial dependencies.

4.5. Incorporating path-choice components into neural networks with knowledge-based initialization (component ④)

To account for the influence of dynamic metro service-network structures (e.g., station or section closures and the operation of short-turning services) on passenger path choice, this component integrates a Logit-based component into the main neural network and utilizes domain-knowledge-informed parameter initialization. The Logit-based component explicitly represents the passenger path-choice process as a differentiable operation within the computation graph. It introduces an input layer, a utility layer, and a Multinomial Logit (Softmax) layer to compute path-choice probabilities for each OD pair. By embedding this logic, the framework transforms path-flow estimation into a transportation-domain-knowledge-guided inference task, helping address the lack of path-level ground truth.

After determining path choice probabilities for OD pairs, we calculate the probability that a path for any given OD pair will traverse each specific section of the network. This section-level probability establishes a direct link to variations in the service network. Changes in the service network directly affect the operating condition of these sections (e.g., section availability and service frequency), which in turn influences path utility and feasible path selection. Therefore, the section traversal probabilities calculated by this module will be utilized by other Knowledge Integrators to model the impact of service network variations on passenger path choice within our predictive framework.

4.5.1. Inputs 5, 6 and 7: service network related information

Input 5 (K^5) provides the domain-knowledge-based initial values for the utility-layer coefficients $\{\beta_\phi\}_{\phi=1}^\Phi$, which enter the utility function in Eq. (22). These initial values are derived from survey-based route-choice studies and are treated as fixed prior values before model training. In configurations where the utility layer is trainable, the corresponding model parameters are initialized from K^5 and then updated during training.

Input 6 ($K_{d,t_0-l^h+i-1}^6$) contains variables $\psi_{\phi,d,t}(p_{\text{od},r})$ used in Eq. (22). These variables describe the characteristics of each candidate path at a specific time point and may vary with the operational state of the network.

Input 7 ($K_{d,t_0-l^h+i-1}^7$) represents a snapshot of the service network state at time $t_0 - l^h + i - 1$. Let $\mathcal{L}_{\text{mtr}} = \{\ell_1, \ell_2, \dots, \ell_{N_{\text{mtr}}}\}$ denote the ordered set of metro lines in operation. The service network state is encoded as a dense directed station-pair-by-line tensor, so the size of $K_{d,t_0-l^h+i-1}^7$ is $|\text{STN}| \times |\text{STN}| \times N_{\text{mtr}}$. The first two dimensions index ordered station pairs (η, μ) , while the third dimension indexes metro lines $\ell \in \mathcal{L}_{\text{mtr}}$. Each element $K_{d,t_0-l^h+i-1}^7(\eta, \mu, \ell)$ represents the departure frequency of trains on line ℓ over the directed adjacent-station section from station η to station μ . If (η, μ) is not a valid adjacent-station section, or if line ℓ does not operate directly over (η, μ) , then $K_{d,t_0-l^h+i-1}^7(\eta, \mu, \ell) = 0$. The opposite travel direction, if present, is represented separately by the entry $K_{d,t_0-l^h+i-1}^7(\mu, \eta, \ell)$.

4.5.2. Constructing path selection components for neural networks

To analyze and incorporate information on passenger path choice into our model, this component first designs a domain-knowledge-integrated sub-network. This sub-network is structured with an Input layer, a Utility layer, and a Multinomial Logit (Softmax) Layer, and is designed to be integrated into the overall deep learning architecture during subsequent processing. The sub-network is jointly trained with the rest of the model and serves to generate knowledge-integrated features used for feature augmentation in the main predictive network.

We have designed a three-layer logit model that is employed to estimate the probability of a passenger selecting each candidate path for a given origin-destination (OD) pair. This model is structured as a neural network, allowing it to be integrated as a component within a broader neural architecture. Its computation proceeds sequentially through three layers, transforming raw path characteristics into choice probabilities. This differentiable structure enables the parameters of the route choice component to be optimized jointly via backpropagation from the network-level OD prediction loss, thereby refining the assignment logic even in the absence of direct path-level supervision.

The Input Layer serves as the initial data interface, responsible for defining and ingesting the fundamental characteristics of each potential travel path. Given OD pair od , define candidate paths $\mathcal{P}_{\text{od}} = \{p_{\text{od},1}, \dots, p_{\text{od},k}\}$. When the OD pair od is clear from context, we abbreviate $p_{\text{od},r}$ as p_r for notational simplicity. Each path $p_{\text{od},r}$ is associated with a time-indexed feature vector $\psi_{d,t}(p_{\text{od},r})$. For any given time step (d, t) , these feature values are precomputed before the forward pass and treated as fixed inputs during training; across different time steps, however, the service-related components may change with the operational state of the network. For each path

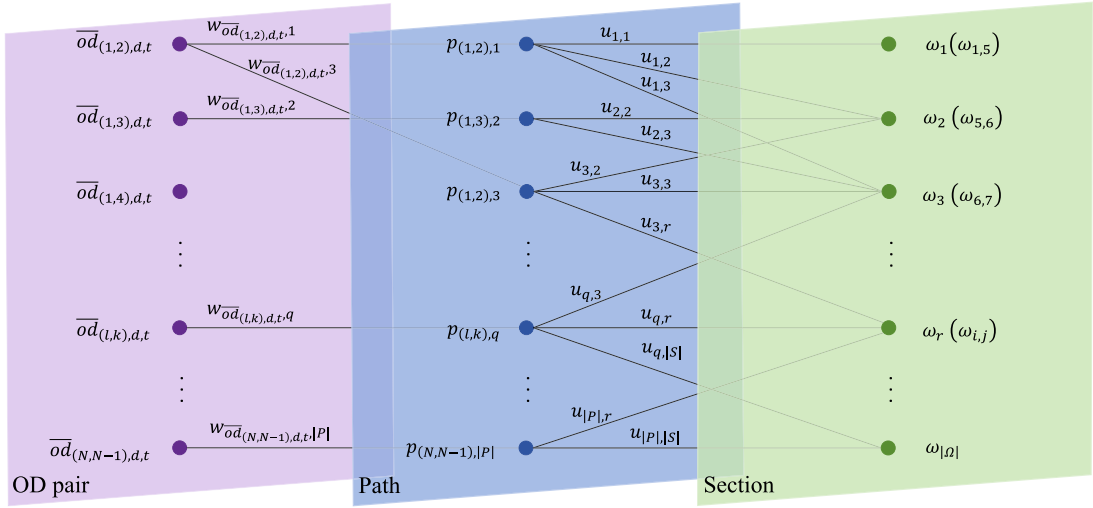


Fig. 5. A tripartite graph illustrating the flow mechanism among OD pairs, paths, and sections.

$p_{\overline{od},r}$, its ϕ th feature $\psi_{\phi,d,t}(p_{\overline{od},r})$, where $\phi \in \{1, 2, \dots, \Phi\}$, is computed by aggregating the corresponding section-level features along the path. In other words, the path feature $\psi_{\phi,d,t}(p_{\overline{od},r})$ aggregates the section-level features along the path, as shown in Eq. (21).

$$\psi_{\phi,d,t}(p_{\overline{od},r}) = \sum_{(\eta,\mu), u_{(p_{\overline{od},r},(\eta,\mu))}=1} \tilde{\psi}_{\phi,d,t,(\overline{od},r),\eta,\mu} \quad (21)$$

where $\tilde{\psi}_{\phi,d,t,(\overline{od},r),\eta,\mu}$ denotes the ϕ -th section-level feature value of section (η, μ) for candidate path $p_{\overline{od},r}$ at time interval (d, t) . The indicator $u_{(p_{\overline{od},r},(\eta,\mu))}$ equals 1 if section (η, μ) belongs to $p_{\overline{od},r}$, and 0 otherwise.

Following the input of an alternative path set $\mathcal{P}_{\overline{od}}$ generated for a given Origin-Destination (OD) pair $\overline{od}$, the Utility Layer calculates the perceived utility of each path. The calculation of the utilities for these path choices $p_{\overline{od},r}$ in set $\mathcal{P}_{\overline{od}}$ is performed using a linear combination of the path's features and their corresponding weights by Eq. (22):

$$U_{\overline{od},r,d,t} = \sum_{\phi=1}^{\Phi} \beta_{\phi} \cdot \psi_{\phi,d,t}(p_{\overline{od},r}) \quad (22)$$

where $U_{\overline{od},r,d,t}$ represents the scalar utility value of path p_r for OD pair $\overline{od}$ at time (d, t) . β_{ϕ} are domain-knowledge-initialized trainable parameters, and are optimized during the training phase to reflect the influence of each feature on route choice decisions.

The final layer is the Multinomial Logit (Softmax) Layer, which converts the calculated utilities into a probability distribution over the set of available paths for an OD pair. The conditional path-choice probability $\Pr(p_r | \overline{od})$ for OD pair $\overline{od}$ is given by Eq. (23):

$$\Pr_{d,t}(p_r | \overline{od}) = \frac{\exp(U_{\overline{od},r,d,t})}{\sum_{p_{\xi} \in \mathcal{P}_{\overline{od}}} \exp(U_{\overline{od},\xi,d,t})} = \frac{1}{1 + \sum_{p_{\xi} \in \mathcal{P}_{\overline{od}}, \xi \neq r} \exp(U_{\overline{od},\xi,d,t} - U_{\overline{od},r,d,t})} \quad (23)$$

The Softmax function normalizes the utilities so that, for any fixed OD pair $\overline{od}$, the resulting probabilities $\Pr_{d,t}(p_r | \overline{od})$ over all candidate paths $p_r \in \mathcal{P}_{\overline{od}}$ sum to one. This layer is a deterministic normalization step and contains no trainable parameters.

The initial values of the utility-layer coefficients $\{\beta_{\phi}\}_{\phi=1}^{\Phi}$ are provided by K^5 , which is derived from a path utility model previously estimated using traffic survey data.

4.5.3. Assessing the probability of an OD pair traversing a specific network section

To estimate which network sections are likely to be used by the flow between each origin and destination, the model combines the estimated path choice probabilities with the network topology to perform the computation.

We employ a tripartite graph (Xu et al., 2019) to illustrate the flow mechanism among OD pairs, paths, and sections, as shown in Fig. 5. The graph is represented as $G' = (\overline{OD} \cup \mathcal{P} \cup \Omega, F \cup U)$, where $\overline{OD}$, $\mathcal{P}$, and Ω denote the sets of OD pairs, paths, and sections, respectively. Here, $F \subseteq \overline{OD} \times \mathcal{P}$ is the OD-path incidence edge set, and $U \subseteq \mathcal{P} \times \Omega$ is the path-section incidence edge set. In other words, F indicates which candidate paths are available for each OD pair, whereas U indicates which network sections belong to each path. The associated path-flow quantity at day d and time interval t is denoted by $f_{\overline{od},p}^{d,t}$. If path p contains section ω , then the indicator $u_{(p,\omega)}$ equals 1; otherwise, it equals 0. For a specific candidate path $p_r \in \mathcal{P}_{\overline{od}}$, this topological relationship is written as $u_{(p_r,\omega)}$.

To determine the usage probability for each network section $\omega \in \Omega$ by travelers from an OD pair $\overline{\text{od}} \in \overline{\text{OD}}$, we map the path selection probabilities generated by the Logit layer onto the physical network topology. The probability that a traveler from OD pair $\overline{\text{od}}$ uses section ω at time (d, t) , denoted $\text{Pr}_{d,t}(\omega \mid \overline{\text{od}})$, is calculated by aggregating the conditional path-choice probabilities of all candidate paths for $\overline{\text{od}}$ that pass through section ω . This process translates path-level preferences directly into section-level probabilities, expressed in Eq. (24):

$$\text{Pr}_{d,t}(\omega \mid \overline{\text{od}}) = \sum_{p_r \in \mathcal{P}_{\overline{\text{od}}}} \text{Pr}_{d,t}(p_r \mid \overline{\text{od}}) u_{(p_r, \omega)} \quad (24)$$

where $\text{Pr}_{d,t}(\omega \mid \overline{\text{od}})$ is the conditional probability that section ω is used at time (d, t) , given a trip associated with OD pair $\overline{\text{od}}$, $\mathcal{P}_{\overline{\text{od}}} \subseteq \mathcal{P}$ is the set of all alternative paths available for the OD pair $\overline{\text{od}}$, $\text{Pr}_{d,t}(p_r \mid \overline{\text{od}})$ is the conditional path-choice probability derived from the Logit model (Eq. (23)), and $u_{(p_r, \omega)}$ is the indicator, which equals 1 if path p_r contains section ω , and 0 otherwise.

4.6. Incorporating service-network variations and their impacts on passenger path choice modeling (component ⑤)

To capture the time-varying characteristics of the metro service network and their impact on path selection, we incorporate this information into the model by transforming departure-frequency information and section-level path-choice information into knowledge-integrated feature representations that are later concatenated with the primary compressed OD input during node-feature construction.

We propose a two-stage approach to construct these knowledge-integrated features. In the first stage, we construct two distinct matrices: one representing passenger path choice probabilities, and the other reflecting the operational frequency of the metro system. In the second stage, to overcome the high dimensionality of these matrices, we apply autoencoders to compress them into compact feature vectors.

The initial step involves translating service-network variations and their impacts into a structured matrix format.

To represent passenger path preferences at time (d, t) , we construct a Section Probability Matrix $M_{d,t}^{\text{pl}}(\overline{\text{od}})$ for each retained OD pair $\overline{\text{od}}$. An element $m_{\eta, \mu, d, t}^{\text{pl}}(\overline{\text{od}})$ in this matrix represents the aggregated probability that a journey associated with OD pair $\overline{\text{od}}$ at time (d, t) passes through the network section $(\eta, \mu) \triangleq \omega$, which is the directed link between stations η and μ . These probabilities are derived from the time-indexed path choice model described earlier. The value of $m_{\eta, \mu, d, t}^{\text{pl}}(\overline{\text{od}})$ is given in Eq. (25):

$$m_{\eta, \mu, d, t}^{\text{pl}}(\overline{\text{od}}) = \text{Pr}_{d,t}((\eta, \mu) \mid \overline{\text{od}}) \quad (25)$$

To capture the time-varying service frequency, we construct a Departure Frequency Matrix $M_{d,t}^{\text{df}}(\ell)$ for each metro line $\ell \in \mathcal{L}_{\text{mtr}}$ at time (d, t) . In contrast to the OD-pair-specific path-choice matrix, this matrix is line-specific. An element $m_{\eta, \mu, d, t}^{\text{df}}(\ell)$ quantifies the operational frequency of line ℓ on the directed network section (η, μ) at time (d, t) . Since the ordered pair (η, μ) already encodes the travel direction, this feature is obtained directly from the service-network input $K_{d,t}^7$, as shown in Eq. (26).

$$m_{\eta, \mu, d, t}^{\text{df}}(\ell) = K_{d,t}^7(\eta, \mu, \ell), \quad (26)$$

where $K_{d,t}^7(\eta, \mu, \ell)$ denotes the departure frequency of line ℓ on the directed section (η, μ) . If (η, μ) is not a valid adjacent-station section, or if line ℓ does not operate directly over (η, μ) at time (d, t) , the corresponding entry is zero.

The matrices generated in Stage 1 ($M_{d,t}^{\text{pl}}(\overline{\text{od}})$ and $M_{d,t}^{\text{df}}(\ell)$) are high-dimensional, sparse, and computationally impractical to use directly as features. To overcome this challenge, we employ autoencoders to compress these matrices into compact, low-dimensional feature matrices. This compression follows the same two-phase methodology outlined in Section 4.4.3. First, two separate sets of autoencoders are pre-trained independently to minimize reconstruction error: one set for the time-indexed Section Probability Matrices $M_{d,t}^{\text{pl}}(\overline{\text{od}})$ and another for the time-indexed Departure Frequency Matrices $M_{d,t}^{\text{df}}(\ell)$. This phase ensures their respective encoders learn to capture the most general patterns of path choice and service frequency. Second, these pre-trained encoders are integrated into the main KNOWCAST framework and are fine-tuned jointly with the core predictive model. During this joint training, their parameters are updated via backpropagation from the final prediction loss. This fine-tuning stage optimizes the encoders to selectively preserve only the information that is most relevant to the forecasting task. The resulting low-dimensional embeddings, $M_{d,t}^{\text{pl,auto}}(\overline{\text{od}})$ and $M_{d,t}^{\text{df,auto}}(\ell)$, thus serve as enhanced, task-specific feature inputs at time (d, t) .

The final step is to integrate these compact feature representations. For each retained OD pair $\overline{\text{od}}$ at time (d, t) , we construct a knowledge-integrated feature representation, denoted by $F_{\text{od}, d, t}^{\text{KI}}$. This representation contains only the knowledge-integrated features associated with that OD pair and does not include the primary OD flow itself. In the present component, it is formed by concatenating the compressed path-choice representation with the complete set of compressed service-state representations of all metro lines at the same time point.

The knowledge-integrated feature representation for a specific OD pair $\overline{\text{od}}$ is constructed by Eq. (27).

$$F_{\text{od}, d, t}^{\text{KI}} = [M_{d,t}^{\text{pl,auto}}(\overline{\text{od}})] \oplus \underbrace{\left[M_{d,t}^{\text{df,auto}}(\ell_1) \oplus M_{d,t}^{\text{df,auto}}(\ell_2) \oplus \dots \oplus M_{d,t}^{\text{df,auto}}(\ell_{N_{\text{mtr}}}) \right]}_{\text{Shared line-level service-state features at } (d, t)} \quad (27)$$

where $\oplus$ denotes the concatenation operation. The term $M_{d,t}^{\text{pl,auto}}(\bar{\text{od}})$ denotes the compressed OD-pair-specific path-choice representation, while the service-state block is shared across all OD pairs at the same time (d, t) . Accordingly, the line-level service-state representations $\{M_{d,t}^{\text{df,auto}}(\mathcal{L})\}_{\mathcal{L} \in \mathcal{L}_{\text{mtr}}}$ are computed once for each time point and reused for all OD pairs at that time.

4.7. Path flow estimation leveraging OD forecasts and intermediate neural features

We begin with the predicted OD flow for short-term online inference. Under the selective aggregation scheme introduced in Section 4.2.2, path-flow and section-flow estimation is performed only for the explicitly retained compressed destination slots. For each origin station m and each retained compressed slot $\tilde{n} \in \{1, \dots, |\text{RDS}|\}$, let $n_m^{(\tilde{n})}$ denote the destination associated with that slot. For a prediction origin (d, t) and forecast horizon $\lambda \in \{1, \dots, I^f\}$, the predicted OD volume associated with this retained slot is defined as

$$\hat{f}_{m, n_m^{(\tilde{n})}}^{(d, t, \lambda)} := \hat{f}_{d, t + \lambda - 1}^A(m, \tilde{n}).$$

The residual aggregated slot $\tilde{n} = |\text{RDS}| + 1$ is excluded from subsequent path-flow and section-flow estimation because it does not correspond to a single destination or a unique candidate path set.

From the Logit sub-network described earlier, we obtain the conditional path-choice probabilities $\text{Pr}^{(d, t, \lambda)}(p_r | m, n_m^{(\tilde{n})})$, i.e., the probability that the OD demand associated with retained slot $\tilde{n}$ for origin m is routed along path p_r .

For each retained destination slot $\tilde{n} = 1, \dots, |\text{RDS}|$, the path flow, which is the expected number of trips on a specific route p_r , is calculated by multiplying the retained-slot OD prediction by the probability of choosing that path, as shown in Eq. (28):

$$\text{PathFlow}_{m, n_m^{(\tilde{n})}, p_r}^{(d, t, \lambda)} = \hat{f}_{m, n_m^{(\tilde{n})}}^{(d, t, \lambda)} \cdot \text{Pr}^{(d, t, \lambda)}(p_r | m, n_m^{(\tilde{n})}). \quad (28)$$

To determine traffic flows on individual network sections, we combine the predicted OD volumes in the retained compressed destination slots with the corresponding path-choice probabilities. Consistent with the path-flow calculation above, the residual aggregated slot is not included in the section-flow calculation. The flow on a specific directed segment from station η to station μ , for prediction origin (d, t) and forecast horizon λ , is denoted by $\text{LinkFlow}_{d, t, \lambda, \eta, \mu}$. It is computed by summing over all origins and all explicitly retained destination slots, as shown in Eq. (29):

$$\text{LinkFlow}_{d, t, \lambda, \eta, \mu} = \sum_{m \in \text{STN}} \sum_{\tilde{n}=1}^{|\text{RDS}|} \hat{f}_{m, n_m^{(\tilde{n})}}^{(d, t, \lambda)} \sum_{p_r \in \mathcal{P}_{m, n_m^{(\tilde{n})}}} \text{Pr}^{(d, t, \lambda)}(p_r | m, n_m^{(\tilde{n})}) u_{(p_r, \eta, \mu)} \quad (29)$$

For each forecast horizon λ , the section-flow matrix is first computed for each evaluation prediction origin (d, t) , and then averaged over all evaluation prediction origins to obtain $\bar{f}_{\eta, \mu}^{(\lambda)}$, as shown in Eq. (30).

$$\bar{f}_{\eta, \mu}^{(\lambda)} = \frac{1}{|\mathcal{E}_{\text{eval}}|} \sum_{(d, t) \in \mathcal{E}_{\text{eval}}} \text{LinkFlow}_{d, t, \lambda, \eta, \mu} \quad (30)$$

where $\mathcal{E}_{\text{eval}}$ denotes the set of evaluation prediction origins (d, t) , and $|\mathcal{E}_{\text{eval}}|$ is its cardinality.

4.8. Deep spatio-temporal learner using graph attention gated recurrent units (GATRU)

To capture spatial-temporal dependencies in metro ridership and fully leverage the components that Knowledge Integrators generate, we design a learner based on the Graph Attention-based Gated Recurrent (GATRU) Units. The GATRU unit combines a Graph Attention Network (GAT) and a Gated Recurrent Unit (GRU) (Dey and Salem, 2017). The GAT aggregates neighbor information into station-level features, while the GRU captures the temporal evolution of ridership patterns and generates hidden states for all nodes over time.

The metro system is modeled as a directed graph $G = (V, \mathcal{E})$, following prior work (Chen et al., 2020; Zhang et al., 2023a; Zong et al., 2025). V denotes the set of vertices representing metro stations, with $|\text{STN}|$ stations in total, and $\mathcal{E}$ denotes the set of directed physical connections between stations. To support graph-attention computation, we define a binary adjacency matrix $M^{\text{adj}} \in \mathbb{R}^{|\text{STN}| \times |\text{STN}|}$, where $M^{\text{adj}}(m, n) = 1$ if $(m, n) \in \mathcal{E}$, and 0 otherwise. This adjacency matrix is used to construct the edge-index representation of the metro topology and to define the feasible attention neighborhood for each station. In the GAT layer, the adjacency structure does not prescribe fixed propagation weights. Instead, it masks the feasible station-to-station connections, while the relative contribution of each neighboring station is learned through attention coefficients normalized over the corresponding neighborhood.

To model spatiotemporal dependencies in metro ridership, we compute attention-weighted features using a Graph Attention Network (GAT) and feed the results into a Gated Recurrent Unit (GRU) to capture temporal dynamics.

Let the input data at time interval (d, t) be represented by $\chi_t = \{\chi_t^1, \chi_t^2, \dots, \chi_t^{|\text{STN}|}\}$. For the GATRU, each station m is treated as a node. Because the predictor operates in the compressed destination space introduced in Section 4.2.2, the node feature for origin m is also constructed in that compressed space. Specifically, let $\mathbf{I}_{d, t}(m, \cdot)$ denote the m th row of the compressed primary OD matrix at time interval (d, t) , whose entries are ordered by the fixed compressed destination slots $\tilde{n} = 1, \dots, |\text{RDS}| + 1$. For each retained slot $\tilde{n} = 1, \dots, |\text{RDS}|$, the corresponding knowledge-integrated feature block is taken from the OD-pair-level knowledge-integrated

feature representation $F_{od,d,t}^{KI}$ associated with the retained real OD pair $(m, n_m^{(\tilde{n})})$. These slot-aligned knowledge-integrated feature blocks are stacked in the same fixed compressed-destination order and concatenated with $\mathbf{I}_{d,t}(m, \cdot)$ to form the node feature χ_t^m . Thus, the primary compressed OD input and the knowledge-integrated feature representations are combined only at this node-feature construction stage. The residual aggregated slot $\tilde{n} = |\text{RDS}| + 1$ remains an aggregate destination channel and is not interpreted as a single real destination. Accordingly, χ_t^m represents the compressed destination distribution of trips originating from station m , together with the aligned knowledge-integrated feature blocks associated with the retained destination slots.

For each node m , we apply multi-head attention to aggregate information from its neighbors $\mathcal{N}(m) \cup \{m\}$. The attention-weighted feature is computed by Eq. (31):

$$f(\chi_t^m) = \text{CONCAT}_{k=1}^{K_{\text{head}}} \left(\text{Act}_{\text{nl}} \left(\sum_{n \in \mathcal{N}(m) \cup \{m\}} \alpha_{mn}^k W_{\text{gat}}^k \chi_t^n \right) \right), \quad (31)$$

where $\mathcal{N}(m)$ denotes the set of stations that are allowed to send messages to station m under the edge-index representation; α_{mn}^k is the attention coefficient from station n to station m for the k -th head, normalized over $\mathcal{N}(m) \cup \{m\}$; W_{gat}^k is the trainable linear transformation matrix associated with the k -th attention head; and $\text{Act}_{\text{nl}}(\cdot)$ denotes a nonlinear activation function.

The resulting features $f(\chi_t^m)$ are then passed into a GRU unit to capture temporal dependencies. The GRU updates are as shown in Eq. (32):

$$\begin{aligned} r_t^m &= \sigma(W_r[h_{t-1}^m, f(\chi_t^m)] + b_r), & z_t^m &= \sigma(W_z[h_{t-1}^m, f(\chi_t^m)] + b_z), \\ \tilde{h}_t^m &= \tanh(W[\tilde{r}_t^m \odot h_{t-1}^m, f(\chi_t^m)] + b), & h_t^m &= (1 - z_t^m) \odot h_{t-1}^m + z_t^m \odot \tilde{h}_t^m \end{aligned} \quad (32)$$

where $\sigma(\cdot)$ denotes the sigmoid activation function, W_r, W_z, W and b_r, b_z, b are trainable weights and biases, h_{t-1}^m is the hidden state of node m from the previous time step, and $\odot$ represents the Hadamard product (element-wise multiplication). The updated hidden state h_t^m integrates both historical context and current attention-based spatial features.

The output of the GATRU for all stations at time t is generated by Eq. (33):

$$H_t = \{h_t^1, h_t^2, \dots, h_t^{|\text{STN}|}\} = \text{GATRU}(\chi_t, H_{t-1}), \quad (33)$$

where $\chi_t = \{\chi_t^1, \chi_t^2, \dots, \chi_t^{|\text{STN}|}\}$ represents the complete set of input node features across the network at time t , and H_t represents the temporally updated hidden states that incorporate both spatial and temporal information.

5. Numerical experiments

This section evaluates the performance of the proposed framework and analyzes how different Knowledge Integrators contribute to forecasting accuracy. First, we compare our model against 10 state-of-the-art benchmarks to validate its overall effectiveness, as detailed in Section 5.2. Second, to investigate the specific contribution of each Knowledge Integrator, we conduct an ablation study using two distinct learners: the proposed GATRU and a baseline GConvLSTM. We conduct a broader ablation suite on the GATRU-based framework (Cases 2b-8), and then examine the transferability of selected Knowledge Integrators under a GConvLSTM-based learner (Cases 9-13). It allows us to isolate the impact of different knowledge integration strategies (see Section 5.3). Finally, we examine how variations in prediction tasks and dataset characteristics affect forecasting outcomes, with results discussed in Section 5.6.

All experiments were conducted on a workstation with two Intel(R) Xeon(R) Gold 5318Y processors @ 2.10 GHz, providing 48 physical cores and 96 logical CPU threads, with 503 GiB of total memory.

5.1. Experimental settings

This section outlines the experimental setup used to validate the KNOWCAST framework on a real-world dataset. We detail the datasets and feature generation processes, the metrics for performance evaluation, and the parameter initialization and tuning strategies employed.

5.1.1. Datasets

To validate the proposed framework, we utilize large-scale Automated Fare Collection (AFC) data from a major metropolitan transit system in Eastern China. Due to confidentiality agreements with the data provider, the specific city and network are anonymized and hereafter referred to as City S. This dataset includes 15-min interval data from City S Metro OD passenger flow, covering Lines 1, 2, 3, 4, the branch line of Line 4, and Line 5. The network topology consists of 154 stations, corresponding to 23,716 (154×154) potential OD-matrix entries, including diagonal entries. The training set time range is from March 1st, 2023, to March 31st, 2023. Although we employ the Selective Aggregation technique to focus on dominant flows (Retention Ratio ≈ 0.23), the training set still comprises over 16 million effective spatiotemporal data points, providing a high-density signal for model training. To better reflect real deployment conditions and reduce the risk of temporal overfitting, we evaluate the model on a later period separated from the training set by a distinct temporal gap, with April excluded between the training and evaluation periods. Consequently, the validation set is drawn from May 1st, 2023 to May 10th, 2023, and the test set from May 11th, 2023 to May 20th, 2023. This dataset captures the operational heterogeneity typical of real-world metro systems, including both scheduled supply-side adjustments and demand-side fluctuations. It documents routine weekday-weekend transitions, dynamic headway adjustments (e.g., on Line 1 and Line 4), and significant

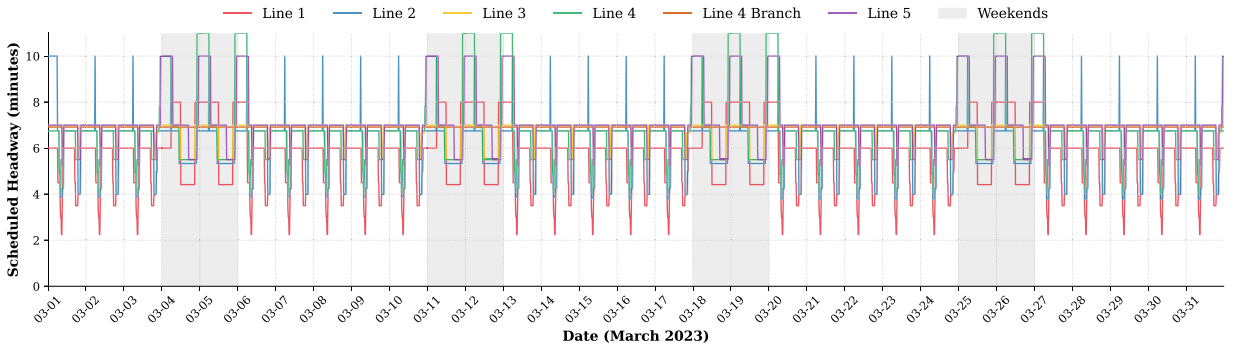


Fig. 6. Dynamic headway adjustments across representative City S Metro lines during March 2023, illustrating the non-stationary supply-side operational environment. Shaded areas indicate weekends.

demand surges during the Labor Day holidays. For instance, as visualized in Fig. 6, the scheduled operational headways fluctuate significantly across different lines and times of day. During peak hours, the headway drops to as low as 2.25 min on the heavily loaded Line 1, while maintaining considerably wider intervals (e.g., 6.5 to 7 min) on newer lines such as Line 5. This pronounced non-stationary supply-side environment justifies the post-processing procedure used in the subsequent section-load evaluation. These planned operational shifts provide a rigorous, non-stationary environment for evaluating model robustness. This split is intended to evaluate temporal generalization under a more realistic deployment setting. The full-month gap between the training period (March) and the validation/test periods (May) reduces the effect of short-term temporal continuity and makes the evaluation more sensitive to whether the model has learned transferable OD-flow patterns under changing seasonal and operational conditions.

The metro timetable data were obtained from the official website of City S Metro. Information regarding station closures and adjustments in departure frequencies was sourced from the service notices page. Population and demographic data were collected from publicly available datasets. The City S Metro network topology in this study, while not as complex as those in megacity systems, features a significant number of interchanges that enable path choice behavior. The 154 stations are distributed across Lines 1, 2, 3, 4, the branch line of Line 4, and 5. These lines are interconnected by 17 distinct transfer stations, creating multiple, non-trivial alternative routes for many Origin-Destination pairs. This network structure validates the necessity of our path choice component (Component ④) for accurately modeling passenger flow distribution.

5.1.2. Feature generation

The explanatory variables detailed conceptually in Section 4.3.1 were operationalized for the City S study area using available data sources and estimation techniques. A data fusion approach was necessary, as fine-grained census data at the station level was unavailable.

The core methodology involved fusing multi-source data. We utilized the localized Seventh National Population Census data of City S, which provides detailed demographic information (e.g., population counts and educational attainment) at the district level. Concurrently, we incorporated high-resolution national population raster density data and mapped each metro station to its corresponding administrative district. The key challenge was the coarse spatial resolution of the census data. We therefore used the finer-grained raster data as the basis for spatially disaggregating the district-level census features (like education profiles) onto smaller geographic units. This fusion process enabled the calculation of detailed population characteristics within the 800-meter catchment area of each station. Specifically, this involved two estimations. First, the catchment population size (PopAct_m) for each station m was estimated via geospatial analysis, aggregating the high-resolution raster population counts within an 800-m circular buffer (representing pedestrian accessibility). Second, structural demographic characteristics were derived from the district-level census bulletin, including the working-age population percentage ($\text{AgePct}^{\text{work}}$), the distribution across four educational attainment levels (primary, junior middle, senior high, tertiary, collectively $\text{EduPct}_{\text{EDU}}$), and the average household size (HHSIZE). These district-level proportions were then spatially allocated to the station's catchment population using the raster-based disaggregation method, yielding the final station-level estimates ($\text{AgePct}_m^{\text{work}}$, $\text{EduPct}_{m,\text{EDU}}$, HHSIZE_m).

Temporal features were more directly extracted. Seasonality indicators (Season_d), a binary weekpart indicator distinguishing weekdays from weekends/holidays (Wknd_d), and the continuous time-of-day variable (TOD_t) were derived directly from the timestamps associated with each AFC transaction record. Event impact scores ($\text{Event}_{m,d,t}$) were constructed based on information regarding major public events (such as concerts, sports games, or large exhibitions) gathered from official news releases and city event calendars. Scores, typically indexed between 0 and 1, were assigned to nearby stations for relevant time intervals, considering factors like the event's scale (e.g., estimated attendance) and the station's proximity to the venue.

These generated socio-demographic and temporal features constitute the input vector $\mathbf{k}_{m,d,t}^{TG}$ used to inform the pre-trained model within Component ②, providing contextually relevant information for trip generation estimation.

5.1.3. Evaluation metrics

To evaluate accuracy, three widely used error metrics are defined: Mean Absolute Error (MAE), Weighted Mean Absolute Percentage Error (WMAPE), and Root Mean Square Error (RMSE), as shown in Eqs. (34)-(36).

Table 2

Summary of Bayesian optimization search space and the final optimized hyperparameter values.

Hyperparameter	Search Range or Choices	Optimized Value
OD Matrix Dimension Retention Ratio	0.23 (fixed)	–
α_{decay} (Decay Rate)	{0.01, 0.05, 0.1, 0.2, 0.5} (categorical)	0.1
Base learning rate	$[1 \times 10^{-5}, 1 \times 10^{-2}]$ (log-uniform)	0.00653
Dropout probability	[0.0, 0.5] (uniform)	0.442
RNN hidden units	{64, 96, 128, 192}	96
GAT feature dimension	{256, 512, 768}	256
Trip generation-informed loss term weight	$[1 \times 10^{-5}, 1 \times 10^{-1}]$ (log-uniform)	0.0264
Additional distribution dim	{2, 3, 4, 5}	5
Additional section feature dim	{5, 6, 7, 8}	6

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|, \quad (34)$$

$$\text{WMAPE} = \frac{\sum_{i=1}^N |y_i - \hat{y}_i|}{\sum_{i=1}^N y_i} \times 100\%, \quad (35)$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}. \quad (36)$$

In Eq. (34), MAE measures the average magnitude of prediction errors in the same units as the target variable. WMAPE in Eq. (35) expresses the error as a percentage of the true values. RMSE in Eq. (36) penalizes larger errors more heavily by squaring deviations before averaging and then taking the square root. In implementation, these metrics are first computed separately for each forecast horizon and then averaged across horizons for reporting.

In the segmented error analysis, we use the Symmetric Mean Absolute Percentage Error (SMAPE) to quantify relative errors across OD groups with different demand levels. For a segment containing N OD-flow observations, SMAPE is defined in Eq. (37) as

$$\text{SMAPE} = \frac{1}{N} \sum_{i=1}^N \frac{|\hat{y}_i - y_i|}{(|y_i| + |\hat{y}_i|)/2 + \epsilon} \times 100\%, \quad (37)$$

where $\epsilon = 10^{-6}$ is a small constant used to avoid division by zero.

5.1.4. Parameter settings and tuning

In urban metro transit path-choice models, two key negative-utility parameters are the per-station disutility (β_{stations}), which expresses the negative utility experienced by travelers for each station traversed on their journey, and the transfer penalty (β_{transfer}), which represents the disutility associated with the effort of making a transfer between lines. Empirical studies estimate $\beta_{\text{transfer}} \approx -0.573$, based on revealed-preference surveys (Mo et al., 2021, 2023). For per-station disutility, however, direct estimates are less commonly reported; this effect is often reflected indirectly through in-vehicle time (IVT) coefficients. By combining the IVT coefficient $\beta_{\text{ivt}} = -0.147$ (per minute) (Mo et al., 2023), with the average station-to-station travel time (approximately 2 min), a reasonable per-station disutility is computed by $\beta_{\text{stations}} \approx -0.147 \times 2 \approx -0.294$, rounded to -0.30 . This reflects that each station stop imposes measurable waiting, boarding/alighting delays, and crowding costs (Mo et al., 2023; Su et al., 2023). Additional studies (Chan et al., 2021; Mo et al., 2020b; Su et al., 2023) corroborate this -0.30 magnitude. Consequently, domain-parameter values are initialized to $\beta_{\text{transfer}} = -0.573$ (per transfer), $\beta_{\text{stations}} = -0.30$ (per station).

Bayesian optimization (Frazier, 2018) was employed to fine-tune several critical hyperparameters, targeting minimization of the validation MAE. We used the automatic hyperparameter optimization framework, Optuna, particularly designed for machine learning. All Optuna studies were conducted using the SQLite storage backend and the Tree-structured Parzen Estimator (TPE) sampler with a MedianPruner for early stopping (Akiba et al., 2019; Bischl et al., 2023). During each trial, model training was restricted to 40 epochs. A total of 80 trials were executed. After completion of the optimization, the best hyperparameter configuration was re-inserted into the original configuration file, and a final full-length training run was performed. The best validation MAE across the 80 Bayesian-optimization trials was achieved at Trial 78.

The hyperparameter search space and the corresponding final optimized values are reported in Table 2. In the main comparative experiments, the OD matrix dimension retention ratio was fixed at 0.23 for all models to ensure that all benchmark, ablation, and full-model comparisons were conducted on the same prediction target and under the same information-retention level. In addition, the convergence behavior of the Bayesian search is illustrated in Fig. 7, where both the MAE at each trial and the running best MAE are plotted against the trial index. Throughout the 80 trials, the TPE sampler consistently proposed promising hyperparameter sets that reduced the validation MAE. Early trials achieved MAE reductions from an initial 1.51 down to the best trial 1.17.

5.2. Comparison with state-of-the-art methods

In comparing the performance of the KNOWCAST framework with other methods, it is important to note that the datasets, metro network structures, and prediction tasks are identical across all methods being compared. We evaluate our proposed framework

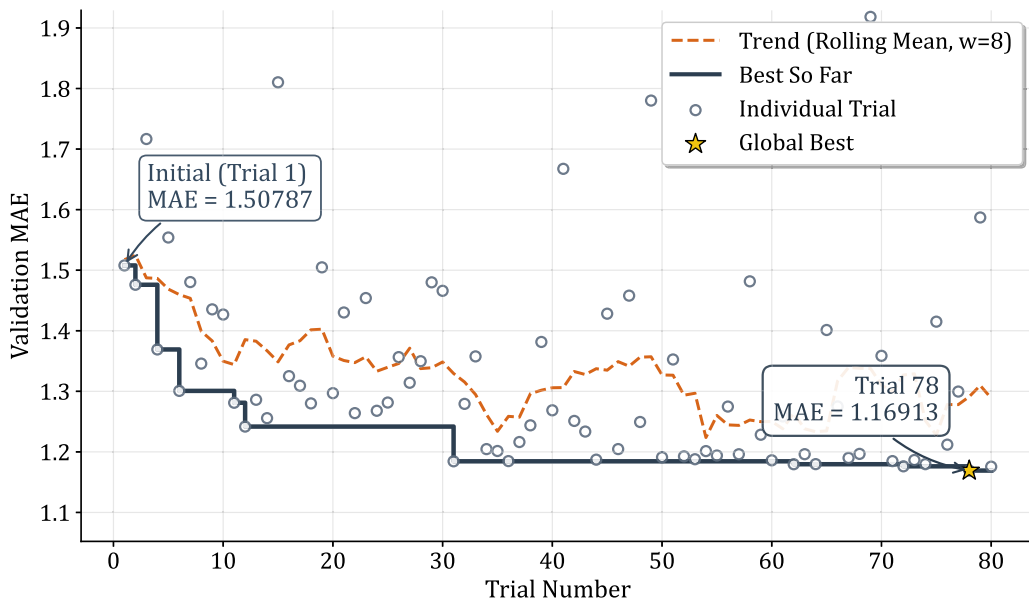


Fig. 7. Bayesian optimization progress.

Table 3

Performance comparison on OD prediction.

Case ID	Model	MAE	RMSE	WMAPE
1	LSTM	1.41	6.48	76.36%
	DCRNN	1.49	8.12	80.69%
	GConvGRU	1.48	7.65	80.15%
	GConvLSTM	1.36	5.61	73.65%
	GCLSTM	1.38	5.95	74.73%
	LRGCN	1.38	5.79	74.73%
	TGCN	1.38	6.04	74.73%
	A3T-GCN	1.46	6.61	79.06%
	AGCRN	1.36	5.61	73.65%
	STConv	1.50	6.95	81.23%
2	Our model	1.02	3.04	54.25%

against several widely recognized spatiotemporal forecasting baselines, including LSTM, DCRNN, GConvGRU, GConvLSTM, GCLSTM, LRGCN, TGCN, A3T-GCN, AGCRN, and STConv. For all these baselines, hyperparameters were tuned via Bayesian optimization (Frazier, 2018) to minimize the validation MAE. Despite this consistent experimental setup, the underlying principles of each method are different. Some methods may not inherently incorporate metro network topology information. Most of the evaluated graph-based models share a common output architecture composed of a Scaled Exponential Linear Unit (SELU) activation function followed by a linear layer.

- Case 1: **LSTM** (Hochreiter and Schmidhuber, 1997) serves as a temporal-only baseline; we compare against **DCRNN** (Li et al., 2017b), **GConvGRU** and **GConvLSTM** (Seo et al., 2018), **GCLSTM** (Chen et al., 2022), **LRGCN** (Li et al., 2019), **TGCN** (Zhao et al., 2019), **A3T-GCN** (Bai et al., 2021), **AGCRN** (Bai et al., 2020), and **STConv** (Yu et al., 2017). These methods integrate Graph Convolutional Networks (GCNs) with recurrent units (GRU/LSTM) or attention mechanisms to capture spatiotemporal dependencies.
- Case 2: **KNOWCAST (Our Model)**: The proposed framework as described in Section 3.

The experimental results of the comparative experiment are shown in Table 3.

As a purely temporal model, LSTM is comparatively less capable of processing spatial relationships. It established a baseline performance that is surpassed by a slight majority, but not all, graph-based methods. A3T-GCN and STConv both yielded higher errors than the simple LSTM. For A3T-GCN, this suggests its attention mechanism, designed to focus on important nodes, may have been counterproductive for the diffuse nature of OD dependencies, losing valuable global information. STConv's failure highlights the unsuitability of applying standard convolutions, designed for regular grid-like data, to irregular graph structures. Models that effectively combine graph convolutions with recurrent units have proven more successful. Among the baselines, AGCRN and GConvLSTM were the top performers, achieving the lowest MAE and RMSE values. AGCRN's strength likely comes from its ability to dynamically learn

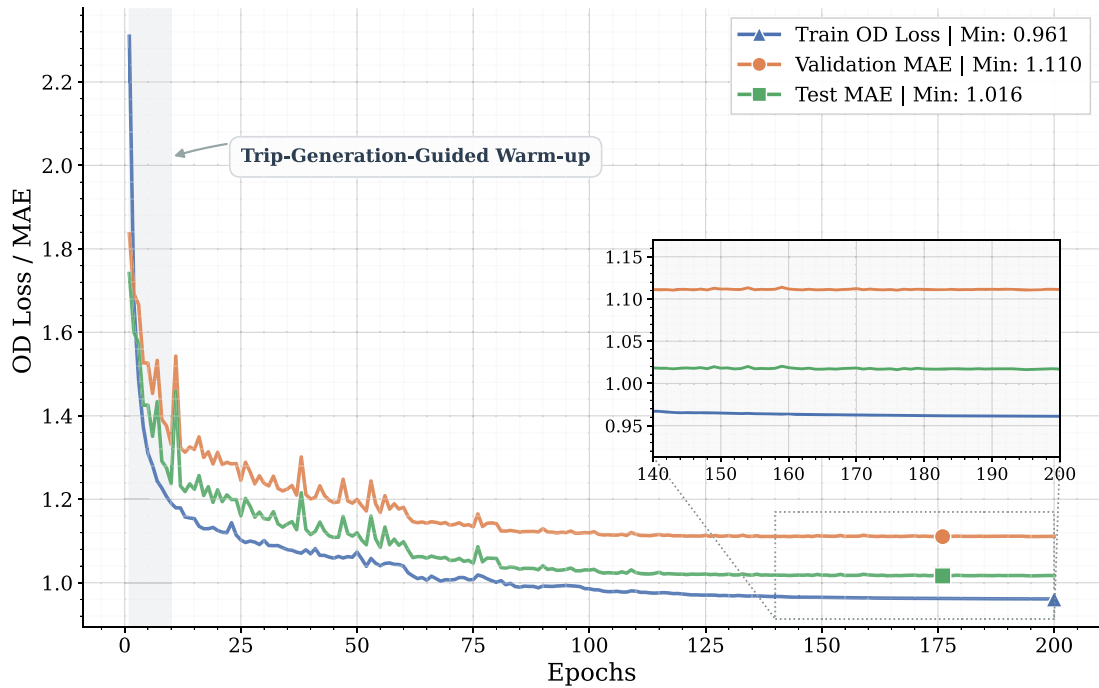


Fig. 8. Convergence behavior over epochs, shown using the training OD loss ($\mathcal{L}_{OD}$) together with the validation and test MAE. The shaded region denotes the first 10 epochs, during which the trip-generation-informed auxiliary weight is active and linearly decays to zero. The magnified inset shows the late-stage stabilization of the three curves. The test curve is shown only for post-hoc diagnostic comparison and is not used for model selection.

the graph structure and capture salient spatial correlations. TGCN achieved a WMAPE of 74.73%, showing competitive performance among GNN-based methods, though slightly higher than GConvLSTM and AGCRN.

Our proposed framework (Case 2) achieves the most competitive performance among all evaluated models, yielding the lowest MAE, RMSE, and WMAPE values in Table 3. Compared with the strongest baseline results, Case 2 reduces MAE from 1.36 to 1.02, RMSE from 5.61 to 3.04, and WMAPE from 73.65% to 54.25%. Case 2 represents the final proposed KNOWCAST framework, integrating the full set of Knowledge Integrators discussed in Section 5.3, including Components ①, ②, ③, ④ with trainable parameters, and ⑤. The detailed contributions of these components, both individually and in smaller groups, are further examined in the subsequent ablation studies. These results indicate that the combination of the GATRU-based Seq2Seq learner and the full set of Knowledge Integrators enables KNOWCAST to capture network topology and traffic dynamics more effectively than the evaluated baselines. The test set is drawn from May, whereas the training set is drawn from March, with April excluded between the two periods. Therefore, the superior test-set performance of Case 2 also provides evidence of temporal generalization under separated evaluation conditions. Despite the temporal separation and changing operational conditions between March and May, the model maintains the lowest test-set error among all evaluated methods (MAE 1.02), suggesting that the framework captures transferable structural patterns rather than merely fitting the temporal regularities of the training month.

Fig. 8 illustrates the convergence behavior of the framework over epochs. The validation and test curves report evaluation MAE, whereas the training curve reports only the primary OD forecasting loss ($\mathcal{L}_{OD}$), excluding the auxiliary trip-generation-informed term ($\mathcal{L}_g$) for scale readability. This treatment is appropriate because $\mathcal{L}_g$ mainly serves as a structural regularizer during the warm-up stage. The shaded Trip-Generation-Guided Warm-up region marks the first 10 epochs, during which the trip-generation-informed weight $w_g(\epsilon)$ remains active and linearly decays to zero under $\alpha_{decay} = 0.1$. This domain-guided weighting scheme provides strong early structural guidance, after which the decayed auxiliary weight shifts optimization toward the primary OD forecasting objective. The magnified inset highlights the late-stage behavior of the three curves, showing that the validation and test MAE remain closely aligned while the training OD forecasting loss also stabilizes. This pattern does not show visible divergence between validation and test performance. The test curve is shown post hoc for analysis only; no early stopping, model selection, or hyperparameter tuning was performed on the test set.

Fig. 9 presents the validation-set performance curves from Case 2, showing how MAE, RMSE, and WMAPE evolve over epochs. Although Case 2 has a lower RMSE on the test set (3.04) than on the validation set (4.18), this difference is mainly attributable to the heavier tail of the validation-set error distribution. The median absolute errors are nearly identical across the validation and test sets (0.253 and 0.260, respectively), whereas the validation set contains larger extreme errors, with a 99.99th percentile error of 126.78 compared with 90.51 on the test set, and a maximum absolute error of 1441.06 compared with 824.06. Because RMSE quadratically penalizes large deviations, these extreme validation-set errors disproportionately increase the validation RMSE. Thus, the lower test

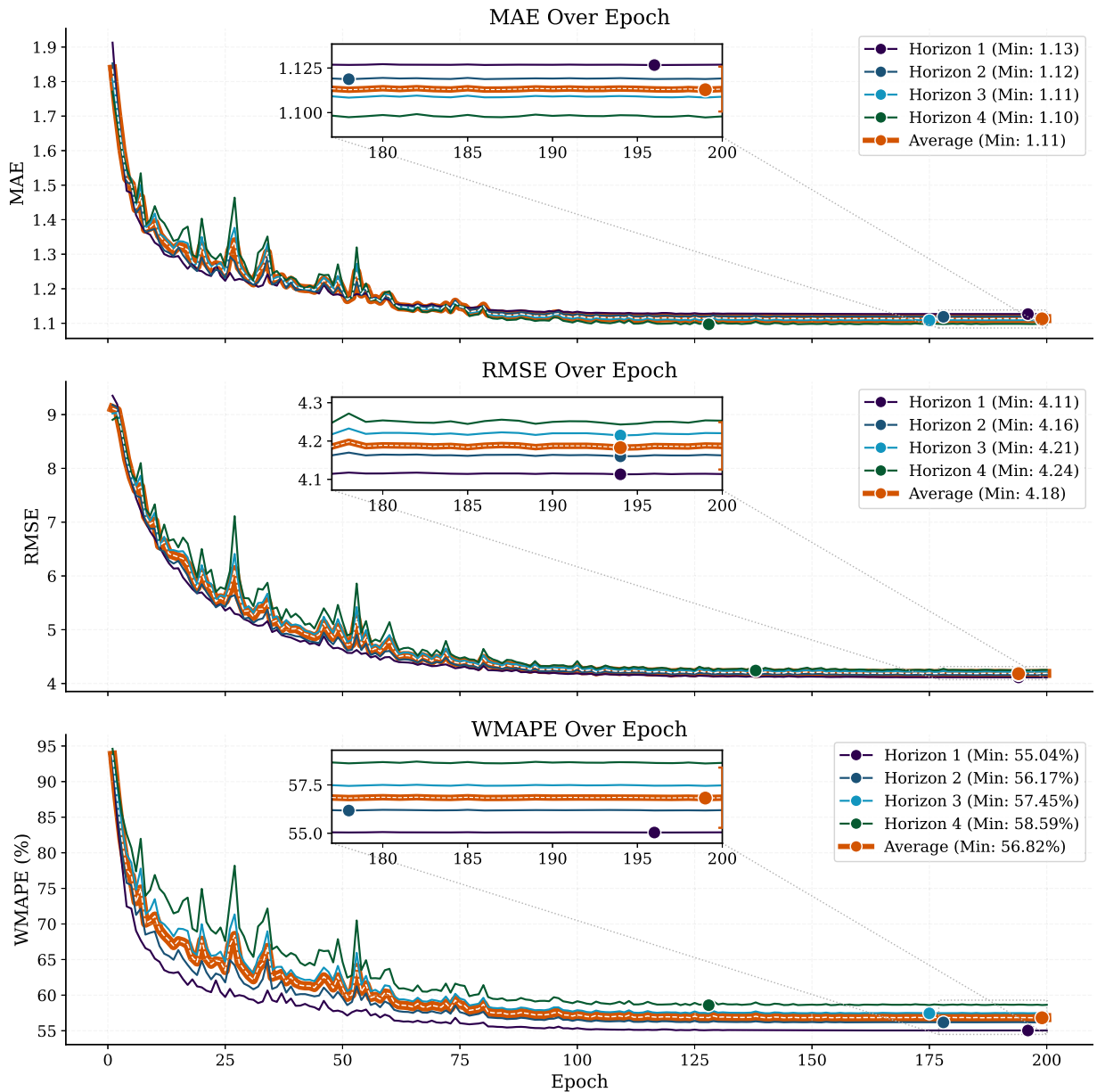


Fig. 9. Validation MAE, RMSE, and WMAPE over epochs.

RMSE does not indicate that the test set is generally easier; rather, it reflects the smaller magnitude of extreme long-tail errors in the test-set distribution.

Fig. 10 illustrates the multi-horizon forecasting results for Case 2 across representative retained OD pairs in the test set. The displayed examples include several dominant high-volume OD pairs. With the multi-step predictions aligned to their actual time of occurrence, the forecast trajectories generally follow the observed temporal patterns across different prediction horizons. Local discrepancies are mainly observed around short-lived demand peaks, where the model tends to produce smoother and more conservative estimates than the abrupt ground-truth fluctuations. According to the exported OD-pair summary aggregated over the displayed horizons, the global MAPE of the displayed OD pairs ranges from 34.28% to 42.80%. Overall, these examples show that the model captures the main temporal patterns of high-volume OD pairs, while some short-lived local demand peaks are estimated more conservatively.

To further evaluate the robustness of the proposed model (Case 2) under the long-tail demand distribution, we conducted a segmented error analysis. This analysis reports group-wise errors for the Head, Body, and Tail OD groups, allowing performance on sparse OD pairs to be checked separately from the aggregate metrics. Using a Lorenz-curve approach (Fig. 11a), the specific OD pairs were partitioned into three groups according to their cumulative contribution to the total passenger flow: the High-volume

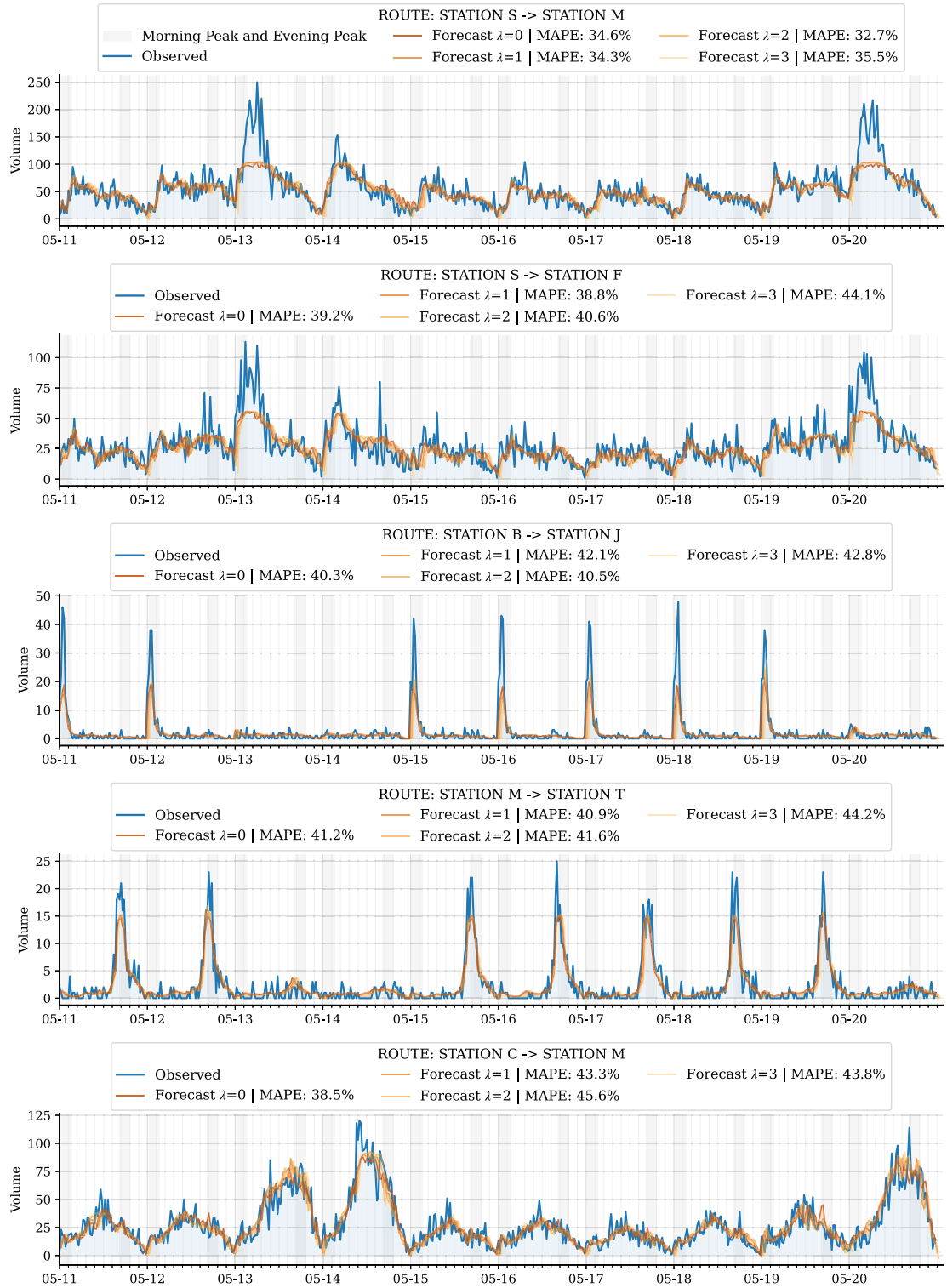


Fig. 10. Comparison of true and multi-horizon predicted OD time series for selected pairs.

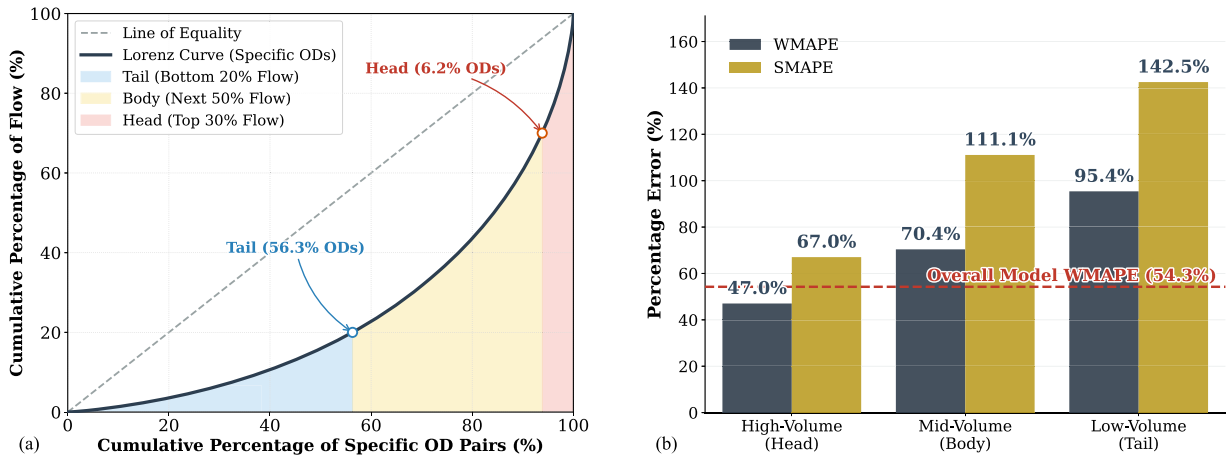


Fig. 11. Segmented error analysis of OD flows. (a) The Lorenz curve illustrating the long-tail distribution, partitioned into High-volume (Head), Mid-volume (Body), and Low-volume (Tail) groups based on cumulative contribution to total passenger flow. (b) Relative error metrics (WMAPE and SMAPE) across the three groups.

(Head) group, which accounts for the top 30% of total flow and contains 6.2% of the OD pairs; the Mid-volume (Body) group, which accounts for the next 50% of total flow and contains 37.5% of the OD pairs; and the Low-volume (Tail) group, which accounts for the bottom 20% of total flow and contains 56.3% of the OD pairs. For the High-volume (Head) OD pairs, which represent the primary operational demand of the network, the model yields a WMAPE of 47.0% and an SMAPE of 67.0% (Fig. 11b). As the inherent flow volume decreases, the relative percentage errors increase: the Mid-volume (Body) group yields a WMAPE of 70.4% and an SMAPE of 111.1%, while the Low-volume (Tail) group reaches a WMAPE of 95.4% and an SMAPE of 142.5%. The dashed reference line in Fig. 11b indicates the overall model WMAPE of 54.3%. The larger percentage errors in the Tail are mainly driven by the small-denominator effect; on highly sparse routes, a prediction deviation of only one or two passengers can result in a disproportionately large percentage error. To verify that the model does not over-predict on sparse routes, we further analyzed its behavior across zero-flow instances. Within the Low-volume (Tail) group, there are 4,361,831 instances where the ground-truth passenger flow is zero. During these instances, the model outputs an average predicted flow of 0.10 passengers, with the maximum absolute prediction not exceeding 8.52. Similarly, across the 1,384,986 zero-flow instances in the Mid-volume (Body) group, the average prediction is 0.74 passengers (max: 22.37). Even in the High-volume (Head) group, where base demands are much larger, the average predicted flow remains constrained at 1.75 passengers across 68,337 zero-demand instances (max: 28.52). Overall, this analysis shows that the domain-knowledge-integrated framework maintains stable performance across both high-volume and sparse OD groups.

5.3. Ablation studies: investigating the contributions of different components in KITP

In this section, we conduct an ablation study to assess the contribution of each Knowledge Integrator to model performance.

5.3.1. Investigation of the contribution of the knowledge integrators

As illustrated in Fig. 4, the Knowledge Integrator consists of Components ① (Allocating Unfinished OD Flow and Aggregating the Resulting OD Matrix), Component ② (Formulating a Trip Generation-Informed Loss Term), Component ③ (Generating OD-Distribution-Based Augmented Feature), Component ④ (Incorporating Path-Choice Components into Neural Networks with Knowledge-Based Initialization), and Component ⑤ (Incorporating Service-Network Variations and Their Impacts on Passenger Path Choice Modeling, realized via Comp. ⑤ and Comp. ⑥). The Learner responsible for capturing spatial-temporal dependencies is based on the GATRU unit.

The GATRU-based ablation cases in this subsection include Cases 2b, 2c, 3, 3b, and 4–8. For our experiments, we selectively integrate the Knowledge Integrators proposed in this paper and also include a generic MLP-based variant for comparison. All these GATRU-based cases are equipped with Comp. ①, as it is an indispensable preprocessing component for obtaining near-complete OD values and applying necessary compression to the OD matrix. Case 3 serves as the minimal GATRU-based control case with only Comp. ①. Each case is detailed below with its corresponding design and components.

- **Case 2b (without Trip Generation):** Retains the complete architecture of Case 2 but thoroughly removes Component ② (Trip Generation-Informed Loss).
- **Case 2c (Temporal Only):** Retains the structure of Component ② but removes all socio-demographic features (Input 2), relying solely on temporal variables to isolate the impact of static spatial attributes.
- **Case 3:** This case only employs Comp. ① to allocate and aggregate OD flow.
- **Case 3b (Generic Baseline):** This case employs the GATRU learner with all available exogenous data (timetable, frequency, demographics). However, these inputs are processed via a generic MLP encoder and concatenated.

Table 4

Experimental configurations for ablation study of model structures and components.

Case ID	Struct.	Comp. ①	Comp. ②	Comp. ③	Comp. ④	Comp. ⑤①	Comp. ⑤②
Case 2b	GATRU	Yes	No	Yes	Yes (trainable)	Yes	Yes
Case 2c	GATRU	Yes	Yes (Temp Only)	Yes	Yes (trainable)	Yes	Yes
Case 3	GATRU	Yes	No	No	–	No	No
Case 3b	GATRU	Yes	Replaced by Generic MLP (using inputs of Comp. ②–⑤)				
Case 4	GATRU	Yes	Yes	No	No	No	No
Case 5	GATRU	Yes	No	Yes	No	No	No
Case 6	GATRU	Yes	No	No	No	No	Yes
Case 7	GATRU	Yes	No	No	Yes	Yes	No
Case 8	GATRU	Yes	No	No	Yes (trainable)	Yes	No
Case 9	GConvLSTM	Yes	No	No	–	No	No
Case 10	GConvLSTM	Yes	Yes	No	No	No	No
Case 11	GConvLSTM	Yes	No	Yes	No	No	No
Case 12	GConvLSTM	Yes	No	No	No	No	Yes
Case 13	GConvLSTM	Yes	No	No	Yes	Yes	No

Case ID: Unique identifier for each case.**Struct.:** Structure of the model.**Comp. ①, ②, ③, ④, ⑤:** Different Knowledge Integrators included in the model.**Comp. ⑤①:** A sub-module of Comp. ⑤, which constructs and integrates the Section Probability Matrix. **Comp.****⑤②:** A sub-module of Comp. ⑤, which constructs and integrates the Departure Frequency Matrix.**Table 5**

Numerical results of the ablation experiments.

Case ID	MAE	RMSE	WMAPE
Case 2b (without Trip Generation)	1.02	3.06	54.27%
Case 2c (Temporal Only)	1.02	3.08	54.42%
Case 3	1.24	5.19	67.15%
Case 3b (GATRU + MLP)	1.21	4.85	65.53%
Case 4	1.22	5.12	66.07%
Case 5	1.14	3.64	61.74%
Case 6	1.21	4.20	65.53%
Case 7	1.19	3.92	64.44%
Case 8	1.10	3.45	59.57%
Case 9	1.35	5.58	73.10%
Case 10	1.35	5.57	72.64%
Case 11	1.24	4.41	67.15%
Case 12	1.32	5.32	73.01%
Case 13	1.31	4.66	70.94%

Table 6

Early-stage convergence comparison at epoch 30.

Model Structure	MAE (at epoch 30)	RMSE(at epoch 30)	WMAPE(at epoch 30)
Case 2 (Full Features)	1.138	4.401	58.11%
Case 2b (without Trip Generation)	1.277	5.729	65.20%
Case 2c (Temporal Only)	1.221	5.927	62.39%

- **Case 4:** This case employs a trip generation-based loss function (Comp. ②).
- **Case 5:** This case calculates trip distribution and merges its results into the OD matrix (Comp. ③).
- **Case 6:** This case merges departure frequency information into the OD matrix (integrating Comp. ⑤②).
- **Case 7:** This case merges path-choice probability matrix into the primary OD matrix (integrating Comp. ④ and Comp. ⑤①). The parameters used for calculating path utility are kept non-trainable (fixed).
- **Case 8:** Similar to Case 7, Case 8 computes path-choice probabilities and merges the result into the primary OD matrix (integrating Comp. ④ and ⑤①). In this instance, the parameters used for computing path utility are trainable.

The details of each case study that have been designed for this purpose are provided in the accompanying Table 4. The GATRU-based ablation results for Cases 2b–8 are reported in the corresponding rows of Table 5.

Case 3 yielded an MAE of 1.24, an RMSE of 5.19, and a WMAPE of 67.15%. This reflects the baseline performance of the model. To distinguish the impact of domain structure from mere data addition, Case 3b incorporated the same exogenous data via a generic MLP encoder and improved the MAE to 1.21, RMSE to 4.85, and WMAPE to 65.53%. The margin between Case 2 and this baseline demonstrates that embedding Logit and Gravity logic utilizes data more effectively than simple black-box concatenation.

Case 4, which introduced the trip generation-informed loss function, showed an MAE of 1.22, an RMSE of 5.12, and a WMAPE of 66.07%. Compared to the baseline, the improvements in these final asymptotic metrics are marginal. This subtle difference further corroborates our core finding: the trip generation component mainly serves as a domain-driven structural constraint that provides a spatial prior, stabilizes the optimization process, and accelerates early-stage convergence. Its contribution to the final asymptotic performance ceiling is comparatively limited. To rigorously isolate the contribution of socio-demographic attributes (Input 2) and investigate the underlying mechanisms of the trip generation module, we extend our ablation analysis to the fully integrated KNOW-CAST framework. The impacts of these spatial features on training dynamics are discussed in Case 2b and Case 2c. The convergence behavior of the complete framework is illustrated by comparing Case 2 (reported in Table 3) with Cases 2b and 2c (reported in Table 5). After training, the final performance metrics of the ablated models (Cases 2b and 2c) closely approach those of the full model (Case 2), with MAE stabilizing uniformly at 1.02 and RMSE showing only marginal differences (3.04 to 3.08). This indicates that given sufficient training epochs, the deep graph neural network possesses the capacity to implicitly learn spatial heterogeneity from historical trends, narrowing the gap in final predictive accuracy caused by the removal of static socio-demographic features. However, an analysis of the training dynamics during the early epochs reveals the primary contribution of these static features. Table 6 extracts the performance metrics of the three models at Epoch 30. At this stage, Case 2c (Temporal Only) shows weaker early-stage control over large errors, with its RMSE remaining higher than that of Case 2b (5.927 vs. 5.729). In contrast, the Full Features model (Case 2) achieves a significantly lower RMSE of 4.401. Case 2c exhibits slower early convergence because the model lacks the necessary inputs to differentiate between stations. The Trip Generation loss ($\mathcal{L}_g$) is emphasized during the early training phase, and the Epoch-30 comparison still reflects the resulting early-stage optimization trajectory. Without socio-demographic features, the trip-generation component provides weaker station-specific guidance, which makes it less effective in controlling large early-stage errors. By incorporating socio-demographic factors (Input 2), the complete model is provided with an initial spatial prior. This domain-specific prior acts as an inductive bias that provides a warm start for the network, reducing early-stage training instability.

Case 5 produced an MAE of 1.14, an RMSE of 3.64, and a WMAPE of 61.74% by calculating trip distribution and merging results into the OD matrix. This integration yielded consistent reductions across all three metrics, demonstrating that embedding trip-distribution knowledge helps the model fit both central tendencies and extreme values more precisely. Case 6 recorded an MAE of 1.21, an RMSE of 4.20, and a WMAPE of 65.53%. Although the reduction in MAE relative to Case 3 is modest, the RMSE improvement indicates that adding departure frequency mitigated larger outliers, and the WMAPE decrease confirms that this domain knowledge provides valid contextual information. Case 7 achieved an MAE of 1.19, an RMSE of 3.92, and a WMAPE of 64.44%, showing improvements in all three metrics compared to Case 3. This indicates that incorporating path utility calculation helped reduce the average error (MAE), lower the squared error magnitude (RMSE), and enhance overall percentage accuracy (WMAPE). Among Cases 3–8, which evaluate individual or partial Knowledge Integrator configurations, Case 8 achieved the best performance, with an MAE of 1.10, an RMSE of 3.45, and a WMAPE of 59.57%. By using trainable parameters for path utility computation, the model adapted more flexibly to data patterns, leading to the lowest absolute errors, the smallest large-error magnitudes, and the highest percentage accuracy. Regarding adaptability, Case 8 appeared more stable than the generic Case 3b during schedule-adjustment periods in the test set, which is consistent with the intended role of the integrated service-network modules in translating supply-side changes into path-probability shifts. In general, the Knowledge Integrators significantly enhance predictive performance. Furthermore, despite the structural complexity, the domain-knowledge-based parameter initialization acts as a strong inductive bias, providing a warm start that accelerates convergence and offsets the marginal increase in per-epoch computational load.

In Case 8 (see Fig. 12), path utility is determined by two learned parameters: per-station disutility β_{stations} and transfer penalty β_{transfer} . During an initial warm-up period (Epoch 0–5), adaptive optimizer dynamics cause a slight overshoot toward less negative values as moment estimates stabilize. Once warm-up ends, both parameters enter a common main-descent window. The two parameters leave the main-descent window at different times: breakpoint detection identifies Epoch 55 for β_{stations} (per-epoch decline rate shifts from -0.0199 to -0.00019 ; see Fig. 13), whereas β_{transfer} continues its gradual descent until roughly Epoch 68, when its slope drops from -0.00406 to -0.00004 (see Fig. 13). Thus, although their early dynamics are synchronized, the transfer penalty undergoes a longer, gentler descent before entering fine-tuning. In this final phase, both parameters settle at $\beta_{\text{stations}} \approx -1.25$, $\beta_{\text{transfer}} \approx -0.77$, leaving station stops weighted about 1.6 times more heavily than transfers in the learned utility function. Quantitatively, robust Theil-Sen slope estimates give

$$s_{\text{stations}}^{\text{TS}} = -0.001501, \quad s_{\text{transfer}}^{\text{TS}} = -0.000936,$$

indicating that the epoch-wise slope of β_{transfer} is about 62% of that of β_{stations} over the fitted trajectory.

The observation that the learned station penalty ($|\beta_{\text{stations}}| \approx 1.25$) exceeds the transfer penalty ($|\beta_{\text{transfer}}| \approx 0.77$) suggests a task-specific operational weighting pattern learned by the model for the studied network. This indicates that, within the forecasting objective, the cumulative disutility associated with traversing multiple intermediate stations may be weighted more heavily than the fixed burden of a transfer event. A plausible explanation is that repeated station stops implicitly accumulate several sources of burden, including dwell times, acceleration/deceleration cycles, and the psychological stress associated with frequent interruptions, causing their total contribution to grow faster than the one-time cost of a transfer. While this learned pattern differs from some survey-based priors, it likely reflects the model's prediction-oriented calibration target rather than a direct estimate of population-level behavioral preference. The resulting parameter values should therefore be interpreted as forecasting-calibrated operational coefficients.

5.3.2. Investigation of the interaction between the knowledge integrators and the learner design

Our framework consists of several Knowledge Integrators that process domain knowledge and a GATRU structure designed to handle the spatiotemporal information in OD data. To explore whether these Knowledge Integrators can also be effective within deep

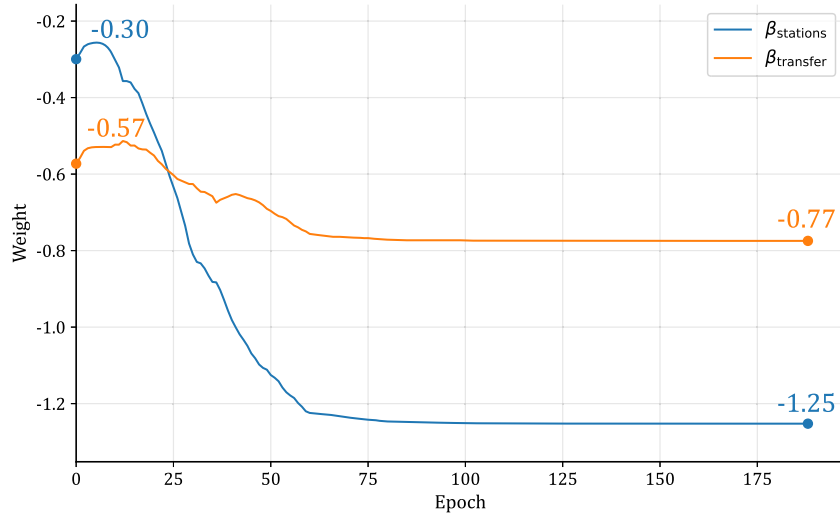


Fig. 12. Evolution of the learned parameters β_{stations} and β_{transfer} over training epochs.

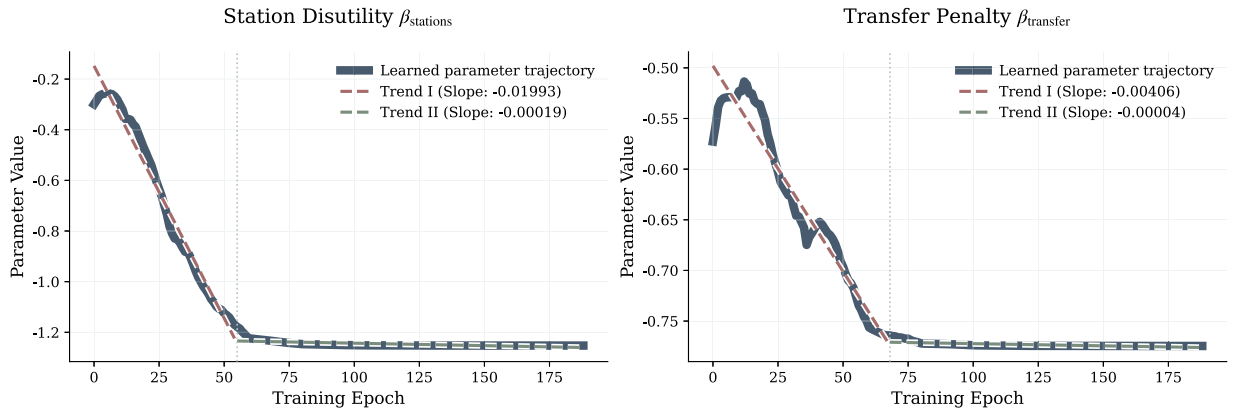


Fig. 13. Breakpoint detection for per-station disutility β_{stations} and transfer penalty β_{transfer} .

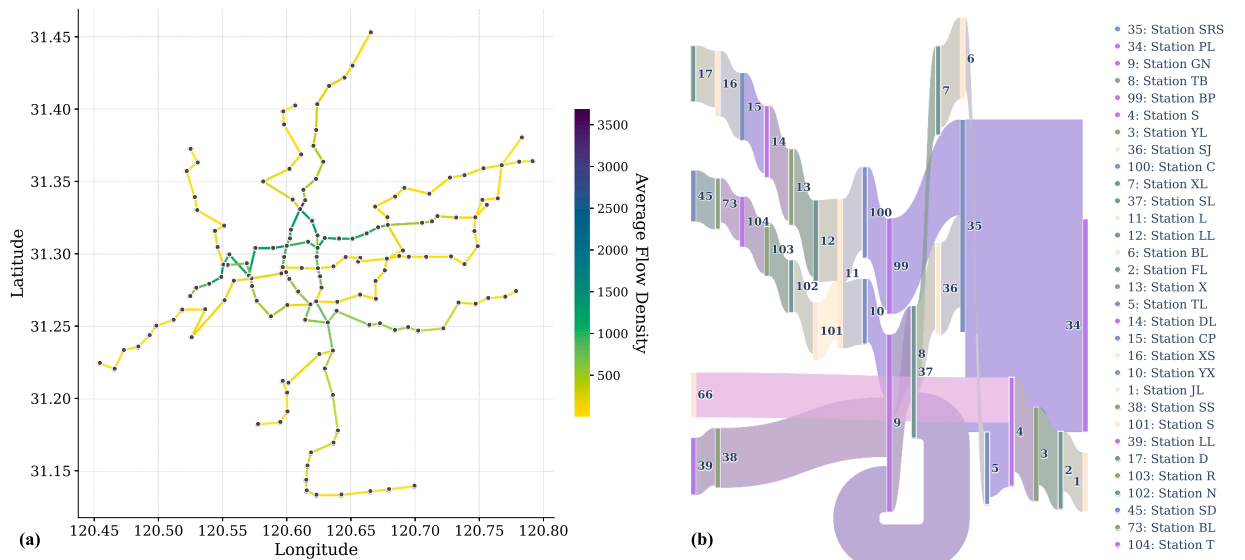


Fig. 14. (a) Spatial distribution of average estimated section-flow density. (b) Sankey diagram of the 33 largest cumulative estimated directed section-flow entries.

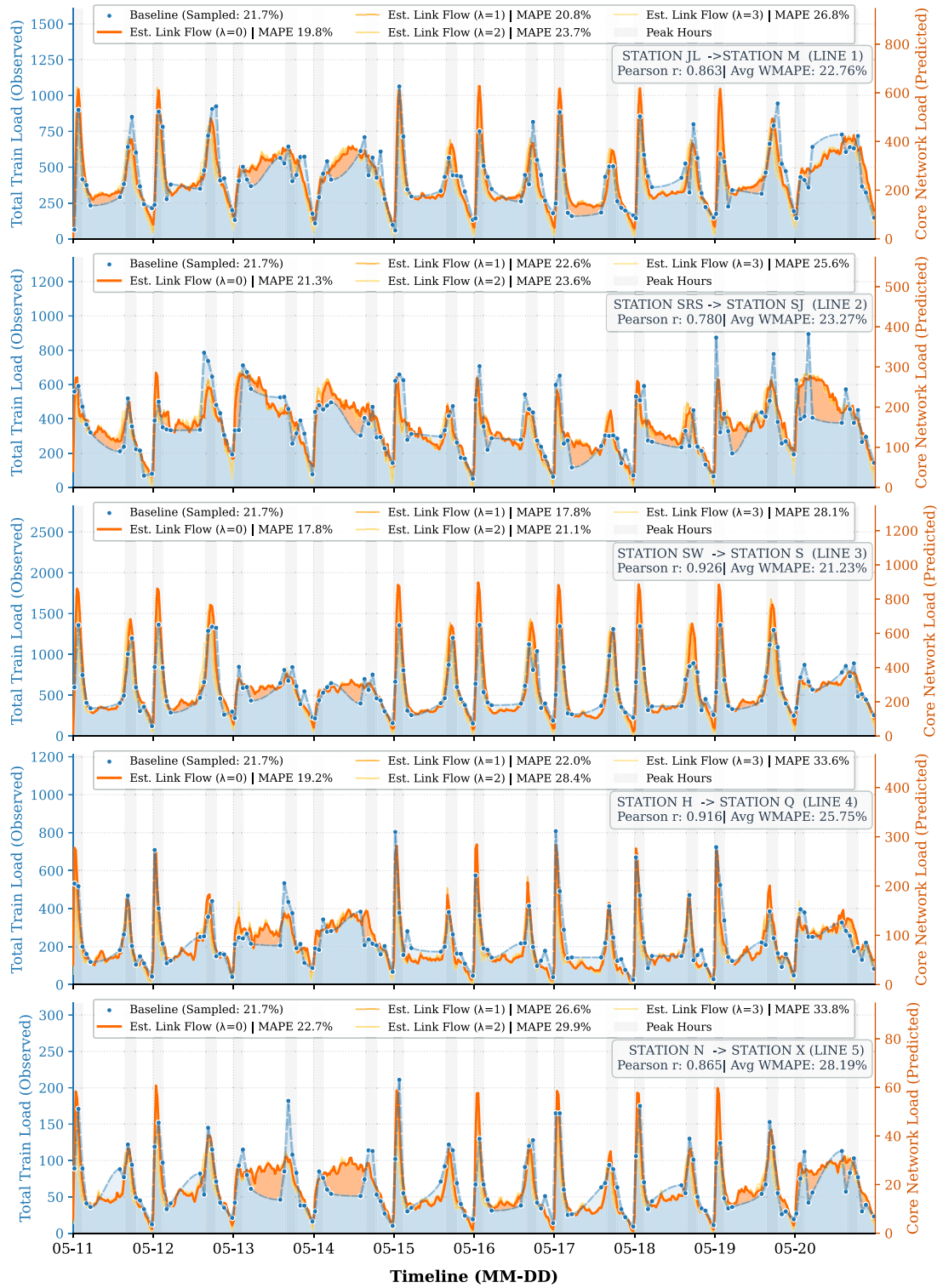


Fig. 15. Time-series comparison of the estimated train loads and the derived train-level observations across five representative congested sections. The actual peak hours are highlighted with gray shading.

Table 7
Empirical comparison of computational complexity.

Case ID	Model Architecture	Avg. Training Time / Epoch (s)	Inference Latency (ms / batch)
Case 9	GConvLSTM	22.5	1250
Case 3	Base GATRU	24.9	1351
Case 3b	GATRU + Generic MLP	28.7	1485
Case 2	KNOWCAST	43.8	2150

Note: The inference latency denotes the execution time for a single forward pass during the validation phase. Measurements represent the average across 5 consecutive epochs.

learning architectures that do not primarily rely on GATRU, we designed the experiments described in this section. All cases from 9 to 13 are equipped with Comp. ①. Each case is detailed below with its corresponding design and components.

- **Case 9:** This case only employs Comp. ① to allocate and aggregate OD flow.
- **Case 10:** This case employs a trip generation-based loss function (Comp. ②).
- **Case 11:** In this case, trip distribution is calculated and its results are merged into the OD matrix (Comp. ③).
- **Case 12:** In this case, departure frequency information is incorporated into the OD matrix (Comp. ⑤).
- **Case 13:** This case computes path utility and path-choice probabilities (employing Comp. ④ and Comp. ⑤). The results are then merged into the primary OD matrix, with the parameters used for calculating path utility kept non-trainable (fixed).

In the case of using a GConvLSTM-based learner, Case 9 yielded an MAE of 1.35, an RMSE of 5.58, and a WMAPE of 73.10%. This represents the baseline performance of the GConvLSTM model without the integration of any domain knowledge or Knowledge Integrators. Case 10 indicated that incorporating the trip generation-based loss function into the GConvLSTM model resulted in an MAE of 1.35, an RMSE of 5.57, and a WMAPE of 72.64%. The most significant improvement for this learner was observed in Case 11, where incorporating trip distribution knowledge (Comp. ③) improved performance to an MAE of 1.24, an RMSE of 4.41, and a WMAPE of 67.15%. The results for Case 12, which integrated departure frequency information, were recorded as an MAE of 1.32, an RMSE of 5.32, and a WMAPE of 73.01%. Compared with Case 9, Case 12 slightly reduced MAE from 1.35 to 1.32 and RMSE from 5.58 to 5.32, while WMAPE remained nearly unchanged, indicating that this specific Knowledge Integrator combination was moderately effective in mitigating larger outliers. Finally, Case 13 demonstrated that incorporating path utility calculations improved performance to an MAE of 1.31, an RMSE of 4.66, and a WMAPE of 70.94%. These results indicate that Knowledge Integrators generally enhance performance across both GATRU-based and GConvLSTM-based learners. Thus, the effectiveness of the Knowledge Integrators is not solely dependent on the underlying learner architecture.

5.4. Computational complexity and efficiency analysis

The computational overhead introduced by the domain-knowledge integration modules is evaluated to ensure the framework's practical feasibility. Table 7 reports the empirical training time per epoch and the inference latency per batch, measured on an Intel Xeon Gold 5318Y CPU workstation.

The baseline models, GConvLSTM (Case 9) and Base GATRU (Case 3), require the least execution time due to their standard recurrent architectures. Processing exogenous data through a generic MLP (Case 3b) introduces a marginal computational load. By contrast, the complete KNOWCAST framework (Case 2) requires approximately 43.8 s per training epoch. This overhead mainly stems from the additional knowledge-based computations, including the explicit matrix operations for candidate path utilities in the Logit sub-network and the gradient computations for the trip-generation-informed loss during backpropagation.

During inference, where loss computations are bypassed, KNOWCAST completes a forward pass in approximately 2150 ms per batch. Although the embedded path-choice routing increases the latency compared to the generic MLP baseline (1485 ms), the measured forward-pass cost remains operationally manageable under a 15-min decision cycle. The empirical results suggest that KNOWCAST preserves operational efficiency while retaining a structurally transportation-consistent formulation.

5.5. Section-flow Illustration and Indirect Operational Plausibility Assessment

Based on the predicted OD flows $\hat{f}_{m,n}^{(d,t,\lambda)}$ and the conditional path-choice probabilities $\Pr^{(d,t,\lambda)}(p_r | m, n)$ obtained from the embedded Logit sub-network, path flows are estimated using Eq. (28), and section flows are then computed using Eq. (29). For visualization, the section-flow matrix is computed for each evaluation prediction origin (d, t) and averaged over all evaluation prediction origins to obtain the average section load $\bar{f}_{\eta,\mu}^{(\lambda)}$, as defined in Eq. (30) and illustrated in Fig. 14(a). To visualize the high-volume section-flow structure implied by the assignment results, a spatial Sankey diagram is constructed from the 33 largest non-zero directed section-flow entries, as shown in Fig. 14(b). In this diagram, nodes represent the stations involved in these high-volume directed sections, and link widths are proportional to the cumulative estimated section-flow volumes between the corresponding upstream and downstream stations.

To assess the operational plausibility of the passenger routing logic learned by the embedded Logit sub-network, an indirect evaluation was conducted. High-confidence continuous section-flow measurements are difficult to obtain because access to the relevant

Table 8

Three-level error tracing and performance metrics for dynamic train loads on representative congested sections.

Representative Section	Line	Level 1 WMAPE (Micro OD)	Level 2 WMAPE (Spatial Aggreg.)	Level 3 WMAPE (Final Load)	Pearson (r)
Station JL → Station M	Line 1	46.87%	11.80%	22.76%	0.863
Station SRS → Station SJ	Line 2	57.57%	19.13%	23.27%	0.780
Station SW → Station S	Line 3	57.49%	19.33%	21.23%	0.926
Station H → Station Q	Line 4	73.13%	37.68%	25.75%	0.916
Station N → Station X	Line 5	73.15%	34.40%	28.19%	0.865

operational systems is restricted. Accordingly, this study derives carriage-level observations through video annotation. The surveillance system provides carriage-level observations; however, because of the substantial annotation workload, manual annotation was restricted to one designated carriage within a selected train for each sampled observation. For comparison with the model-derived section-load estimates, these carriage-level counts were expanded to train-level observations using an expansion factor calibrated for the same corridor and service period. This conversion remains an operational approximation, and residual inter-carriage imbalance, especially during peak periods, is therefore treated as a source of uncertainty. The observations were sampled at fourteen fixed time slices each day: 07:15, 08:00, 08:45, 09:30, 10:15, 11:00, 16:30, 17:15, 18:00, 18:45, 19:30, 20:15, 21:00, and 22:30. Sampling was conducted over 10 days, including weekends, to cover both regular commuter and non-commuter loading conditions.

To compare the model-derived assignment results with the expanded train-level observations, a post-processing alignment procedure was applied. This procedure includes scale compensation, temporal dispersion, and headway-based load conversion. Specifically, because KNOWCAST uses the selective aggregation strategy detailed in Section 4.2.2, the section-flow estimates are computed from the explicitly retained compressed destination slots, while the video observations reflect total passenger volume. A dynamic scaling compensation factor was therefore applied to account for the non-retained destinations represented by the residual aggregated slot. In addition, because assignment outputs do not directly provide the passing time of passengers at downstream sections, a Gaussian convolution kernel was used to temporally disperse the compensated flows, accounting for travel-time delay and passenger temporal dispersion parameterized by topological hop distance. Finally, the dispersed flows were combined with dynamic headway ratios to convert continuous section volumes into train-load estimates. Accordingly, the following comparison should be read as an indirect operational plausibility assessment of the resulting section-load estimates based on aligned train-level observations. It is not a direct validation against continuous section-flow ground truth.

To thoroughly demonstrate the practical efficacy of the assignment module, a three-level error tracing analysis was conducted across five representative bottlenecks on different metro lines (Table 8).

The analysis traces the change in error across the three evaluation levels, from Micro OD prediction to spatial aggregation and then to the final load-level comparison. Level 1 represents the WMAPE of the relevant microscopic OD predictions before aggregation. Level 2 represents the WMAPE after spatial aggregation, where the OD flows associated with the target section are summed into a section-related aggregate OD series. Level 3 represents the final load-level WMAPE after applying the spatiotemporal Gaussian alignment and headway-based load conversion. As shown, the final train load achieves a Weighted Mean Absolute Percentage Error (WMAPE) ranging from 21.23% to 28.19% and Pearson correlation coefficients (r) between 0.780 and 0.926. Furthermore, as illustrated in Fig. 15, the post-processed multi-horizon section-load series maintain a consistent trend with the empirical snapshots. This quantitative decomposition provides supporting evidence that the spatial assignment logic can translate stochastic OD demand into operationally meaningful section-load estimates.

5.6. Impact of Forecasting Task Settings on OD Forecasting Performance

This section examines how different forecasting task settings affect OD forecasting performance. Two groups of settings are considered: the input-output window lengths, represented by l^h and l^f , and the retained dimension used in the selective aggregation of OD matrices. The retained-dimension experiment requires separate interpretation because changing the retention ratio also changes the dimensionality of the prediction target. A more aggressive compression setting aggregates more low-volume destinations into the residual aggregated column, reducing sparsity and generally lowering the resulting error metrics. Therefore, this experiment is reported as a task-setting sensitivity analysis, with the changed prediction target considered.

The findings from each study are as follows:

- When varying the number of historical time intervals l^h and future forecasting intervals l^f , the results show that both settings affect prediction performance. With a short historical input window (e.g., $l^h = 2$, $l^f = 4$), performance degrades because the model has limited historical context, yielding a WMAPE of 61.89%, an MAE of 1.30, and an RMSE of 4.70. A longer forecast horizon also increases task difficulty: when $l^h = 4$ and $l^f = 12$, the WMAPE, MAE, and RMSE increase to 74.55%, 1.60, and 8.00, respectively.
- When varying the retained dimension in the compressed OD representation, more aggressive compression leads to lower error metrics. Retaining 15% of the original target dimension yields a WMAPE of 43.67%, an MAE of 0.86, and an RMSE of 2.65. By contrast, retaining 80% of the original target dimension yields a higher WMAPE of 77.94%, an MAE of 1.72, and an RMSE of 9.41. This result should be interpreted together with the changed prediction target: stronger compression aggregates more sparse, low-volume destinations into the residual aggregated column, which reduces target sparsity, improves computational efficiency, and encourages the model to focus on dominant traffic patterns.

6. Conclusion

In this study, we proposed KNOWCAST, a domain-knowledge-integrated framework for joint short-term network-level OD-flow forecasting and path-flow estimation in urban metro systems. The framework combines transportation domain knowledge with the data-driven strengths of deep learning by embedding knowledge from trip generation, trip distribution, and traffic assignment into a sequence-to-sequence architecture with GATRU as the core learning component. The integration of domain knowledge at both the architectural and parameter-initialization levels improves the framework's practical adaptability and interpretability. The knowledge-informed architecture embeds transportation mechanisms into the prediction process, while knowledge-derived initialization provides selected trainable parameters with behaviorally meaningful starting values that can be refined during training. The same design also supports joint OD-flow forecasting and path-flow estimation. Intermediate representations generated by the knowledge-integrated components are passed to the embedded differentiable assignment layer, allowing path-flow estimates to be inferred from predicted OD demand without path-level ground-truth labels while remaining structurally coupled to the OD forecasts. These inferred path-flow estimates can then be aggregated to obtain section-level flow estimates, providing operationally useful information for metro management. However, direct validation of path-flow estimates remains challenging because of the observability gap in urban metro systems. AFC records reveal entry–exit transactions but do not identify the realized routes, while supplementary sensing technologies such as Wi-Fi or Bluetooth probes often cannot reliably reconstruct complete in-network trajectories. Therefore, the path-flow estimates in this study should be interpreted as structurally constrained estimates rather than directly observed quantities. Their plausibility is supported by the embedded Logit-based assignment structure, which allocates predicted OD demand across topology-feasible candidate paths according to path-choice probabilities.

Our numerical experiments on a real-world dataset from the City S Metro demonstrate the effectiveness of KNOWCAST relative to ten state-of-the-art baseline models across multiple evaluation metrics, including Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and Weighted Mean Absolute Percentage Error (WMAPE). To systematically quantify the accuracy improvements from integrating different Knowledge Integrators, we conducted a series of ablation studies. These studies showed that the Knowledge Integrators contribute in different ways: some directly improve final predictive accuracy, while others primarily enhance convergence stability, reduce large-error outliers, or provide stronger structural guidance during training. The segmented error analysis further confirms the robustness of KNOWCAST under the long-tail demand distribution. The model captures high-density trunk flows while maintaining low non-zero predictions in sparse OD groups. The zero-flow analysis further shows that domain-knowledge integration supports global network reliability without overfitting to low-volume noise. The architecture of KNOWCAST also provides potential for scalability and generalizability. Regarding scalability, KNOWCAST leverages Graph Attention-based Gated Recurrent Units that focus on local neighborhood dependencies, reducing reliance on fully connected station-to-station interactions as network size increases. Regarding generalizability, KNOWCAST is topology-aware because it embeds the logic of trip generation, trip distribution, and traffic assignment. By utilizing socio-demographic covariates, network topology, and assignment constraints as structural inputs, KNOWCAST remains sensitive to changes such as new line openings or revised TAZ-like station catchment definitions. Although the current study validates the model in a single city, this mechanism-informed and modular design suggests potential adaptability to network-evolution scenarios and other urban network topologies. The use of trainable parameters initialized from domain knowledge further strengthens interpretability. These parameters provide behaviorally meaningful starting values for optimization, while their post-training trajectories offer an interpretable record of how the data support, refine, or depart from the original transportation-theory priors. Accordingly, the fine-tuned parameter values should be interpreted in the context of the forecasting task and the studied network, as calibrated operational coefficients rather than direct estimates of universally stable behavioral preferences. More broadly, the knowledge-informed components act as structural regularizers by constraining the optimization toward transportation-consistent solutions and reducing the tendency to overfit low-volume patterns in the training data.

Overall, the results show that integrating transportation domain knowledge into deep learning can improve OD-flow forecasting while enabling structurally consistent path-flow estimation. This integration also supports the development of more interpretable, robust, and data-efficient models for metro network analysis. Although this study validates the overall effectiveness of KNOWCAST, its complex multi-component architecture leaves several issues for further investigation. Future work should conduct more comprehensive sensitivity analyses to quantify the model's dependence on domain-knowledge-based parameter initialization and to examine nonlinear interactions among different Knowledge Integrators. The framework could also be extended to more complex operational scenarios, such as forecasting metro passenger-flow changes during emergency evacuations, to assess and improve model resilience under disruptive conditions.

CRedit authorship contribution statement

Hao Zheng: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Han Zheng:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **George Giannopoulos:** Writing – review & editing, Supervision, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors have no conflicts of interest to declare.

Acknowledgment

This work is supported by the Talent Fund of Beijing Jiaotong University under Grant No. 2024XKRC055.

References

- Ahmed, M.S., Cook, A.R., 1979. Analysis of freeway traffic time-series data by using Box-Jenkins techniques. *Transp. Res. Rec.* 722, 1–9.
- Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M., 2019. Optuna: a next-generation hyperparameter optimization framework. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 2623–2631.
- Anvari, S., Tuna, S., Canci, M., Turkay, M., 2016. Automated Box-Jenkins forecasting tool with an application for passenger demand in urban rail systems. *J. Adv. Transp.* 50 (1), 25–49.
- Bai, J., Zhu, J., Song, Y., Zhao, L., Hou, Z., Du, R., Li, H., 2021. A3T-GCN: attention temporal graph convolutional network for traffic forecasting. *ISPRS Int. J. Geo-Inf.* 10 (7), 485.
- Bai, L., Yao, L., Li, C., Wang, X., Wang, C., 2020. Adaptive graph convolutional recurrent network for traffic forecasting. *Adv. Neural Inf. Process. Syst.* 33, 17804–17815.
- Bischl, B., Binder, M., Lang, M., Pielok, T., Richter, J., Coors, S., Thomas, J., Ullmann, T., Becker, M., Boulesteix, A., 2023. Hyperparameter optimization: foundations, algorithms, best practices, and open challenges. *Wiley Interdiscip. Rev. Data Min. Knowl. Discov.* 13 (2), e1484.
- Blume, S. O.P., Corman, F., Sansavini, G., 2022. Bayesian origin-destination estimation in networked transit systems using nodal in-and outflow counts. *Transp. Res. B Methodol.* 161, 60–94.
- Chan, H.-Y., Chen, A., Li, G., Xu, X., Lam, W., 2021. Evaluating the value of new metro lines using route diversity measures: the case of Hong Kong's mass transit railway system. *J. Transp. Geogr.* 91, 102945.
- Chen, E., Ye, Z., Wang, C., Xu, M., 2019. Subway passenger flow prediction for special events using smart card data. *IEEE Trans. Intell. Transp. Syst.* 21 (3), 1109–1120.
- Chen, J., Liu, L., Wu, H., Zhen, J., Li, G., Lin, L., 2020. Physical-virtual collaboration graph network for station-level metro ridership prediction. *arXiv preprint arXiv:2001.04889*.
- Chen, J., Wang, X., Xu, X., 2022. GC-LSTM: graph convolution embedded lstm for dynamic network link prediction. *Appl. Intell.* 52 (7), 7513–7528. <https://doi.org/10.1007/s10489-021-02518-9>
- Chen, Z., Badrinarayanan, V., Lee, C.-Y., Rabinovich, A., 2018. GradNorm: gradient normalization for adaptive loss balancing in deep multitask networks. In: *International Conference on Machine Learning*. PMLR, pp. 794–803.
- Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y., 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*.
- Dash, T., Chitlangia, S., Ahuja, A., Srinivasan, A., 2022. A review of some techniques for inclusion of domain-knowledge into deep neural networks. *Sci. Rep.* 12 (1). <https://doi.org/10.1038/s41598-021-04590-0>
- Dey, R., Salem, F.M., 2017. Gate-variants of gated recurrent unit (GRU) neural networks. In: *2017 IEEE 60th International Midwest Symposium on Circuits and Systems (MWSCAS)*. IEEE, pp. 1597–1600.
- Frazier, P.I., 2018. A tutorial on Bayesian optimization. *arXiv preprint arXiv:1807.02811*.
- Galliani, G., Secchi, P., Ieva, F., 2024. Estimation of dynamic origin-destination matrices in a railway transportation network integrating ticket sales and passenger count data. *Transp. Res. A Policy Pract.* 190. <https://doi.org/10.1016/j.tra.2024.104246>
- Gazis, D.C., Knapp, C.H., 1971. On-line estimation of traffic densities from time-series of flow and speed data. *Transp. Sci.* 5 (3), 283–301.
- Grindey, G.J., Amin, S.M., Rodin, E.Y., Garcia-Ortiz, A., Kalman filter approach to traffic modeling and prediction. In: *Intelligent Transportation Systems*. SPIE, pp. 234–240.
- Gugaliya, J., Leonard, W., Damer, P., Barabino, M., 2021. Hybrid approach bolsters predictive maintenance. <https://new.abb.com/news/detail/80774/hybrid-approach-bolsters-predictive-maintenance>.
- Guo, J., Huang, W., Williams, B.M., 2014. Adaptive Kalman filter approach for stochastic short-term traffic flow rate prediction and uncertainty quantification. *Transp. Res. C Emerg. Technol.* 43, 50–64.
- Guo, M., Haque, A., Huang, D.-A., Yeung, S., Fei-Fei, L., 2018. Dynamic task prioritization for multitask learning. In: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 270–287.
- Guo, S., Lin, Y., Feng, N., Song, C., Wan, H., . Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 33, pp. 922–929.
- Guo, Y.-q., Wang, X.-y., Xu, Q., Liu, S., Liu, S., Han, J., 2020. Weather impact on passenger flow of rail transit lines. *Civ. Eng. J.* 6 (2), 276–284.
- He, Y., Li, L., Zhu, X., Tsui, K.L., 2022. Multi-graph convolutional-recurrent neural network (MGC-RNN) for short-term forecasting of transit passenger flow. *IEEE Trans. Intell. Transp. Syst.* 23 (10), 18155–18174.
- Heydari, A.A., Thompson, C.A., Mehmood, A., 2019. SoftAdapt: techniques for adaptive loss weighting of neural networks with multi-part loss functions. *arXiv preprint arXiv:1912.12355*.
- Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. *Neural Comput.* 9 (8), 1735–1780.
- Huang, B., Wu, B., Barry, M., 2010. Geographically and temporally weighted regression for modeling spatio-temporal variation in house prices. *Int. J. Geogr. Inf. Sci.* 24 (3), 383–401. <https://doi.org/10.1080/13658810802672469>
- Jiang, J., Lin, F., Fan, J., Lv, H., Wu, J., 2019. A destination prediction network based on spatiotemporal data for bike-sharing. *Complexity* 2019, 7643905. <https://doi.org/10.1155/2019/7643905>
- Jiang, W., Ma, Z., Koutsopoulos, H.N., 2022. Deep learning for short-term origin-destination passenger flow prediction under partial observability in urban railway systems. *Neural Comput. Appl.* 34 (6), 4813–4830. <https://doi.org/10.1007/s00521-021-06669-1>
- Jiao, P., Li, R., Sun, T., Hou, Z., Ibrahim, A., 2016. Three revised Kalman filtering models for short-term rail transit passenger flow prediction. *Math. Probl. Eng.* 2016, 9717582. <https://doi.org/10.1155/2016/9717582>
- Jin, S., Jing, C., Yao, S., Zhang, Y., Zhao, P., Zhang, J., 2025. SastGCN: semantic-augmented spatio-temporal graph convolutional network for subway flow prediction. *Int. J. Appl. Earth Obs. Geoinf.* 139, 104530.
- Ke, J., Qin, X., Yang, H., Zheng, Z., Zhu, Z., Ye, J., 2021. Predicting origin-destination ride-sourcing demand with a spatio-temporal encoder-decoder residual multi-graph convolutional network. *Transp. Res. C Emerg. Technol.* 122, 102858.
- Krizhevsky, A., Sutskever, I., Hinton, G.E., 2017. ImageNet classification with deep convolutional neural networks. *Commun. ACM* 60 (6), 84–90.
- Li, J., Han, Z., Cheng, H., Su, J., Wang, P., Zhang, J., Pan, L., 2019. Predicting path failure in time-evolving graphs. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1279–1289.
- Li, Y., Wang, X., Sun, S., Ma, X., Lu, G., 2017a. Forecasting short-term subway passenger flow under special events scenarios using multiscale radial basis function networks. *Transp. Res. C Emerg. Technol.* 77, 306–328.
- Li, Y., Yu, R., Shahabi, C., Liu, Y., 2017b. Diffusion convolutional recurrent neural network: data-driven traffic forecasting. *arXiv preprint arXiv:1707.01926*.
- Liang, X., Wang, G., Min, M.R., Qi, Y., Han, Z., 2019. A deep spatio-temporal fuzzy neural network for passenger demand prediction. In: *Proceedings of the 2019 SIAM International Conference on Data Mining*. SIAM, pp. 100–108.

- Liu, L., Chen, J., Wu, H., Zhen, J., Li, G., Lin, L., 2020. Physical-virtual collaboration modeling for intra-and inter-station metro ridership prediction. *IEEE Trans. Intell. Transp. Syst.* 23 (4), 3377–3391.
- Liu, L., Qiu, Z., Li, G., Wang, Q., Ouyang, W., Lin, L., 2019a. Contextualized spatial-temporal network for taxi origin-destination demand prediction. *IEEE Trans. Intell. Transp. Syst.* 20 (10), 3875–3887. <https://doi.org/10.1109/tits.2019.2915525>
- Liu, L., Zhu, Y., Li, G., Wu, Z., Bai, L., Lin, L., 2023. Online metro origin-destination prediction via heterogeneous information aggregation. *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (3), 3574–3589. <https://doi.org/10.1109/TPAMI.2022.3178184>
- Liu, S., Johns, E., Davison, A.J., 2019b. End-to-end multi-task learning with attention. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1871–1880.
- Liu, Y., Liu, Z., Jia, R., 2019c. DeepPF: a deep learning based architecture for metro passenger flow prediction. *Transp. Res. C Emerg. Technol.* 101, 18–34. <https://doi.org/10.1016/j.trc.2019.01.027>
- Lu, Y., Ding, H., Ji, S., Sze, N.N., He, Z., 2021. Dual attentive graph neural network for metro passenger flow prediction. *Neural Comput. Appl.* 33, 13417–13431.
- Lv, S., Wang, K., Yang, H., Wang, P., 2024. An origin-destination passenger flow prediction system based on convolutional neural network and passenger source-based attention mechanism. *Expert Syst. Appl.* 238, 121989.
- Ma, X., Dai, Z., He, Z., Ma, J., Wang, Y., Wang, Y., 2017. Learning traffic as images: a deep convolutional neural network for large-scale transportation network speed prediction. *Sensors* 17 (4), 818.
- Ma, X., Zhang, J., Du, B., Ding, C., Sun, L., 2018. Parallel architecture of convolutional bi-directional LSTM neural networks for network-wide metro ridership prediction. *IEEE Trans. Intell. Transp. Syst.* 20 (6), 2278–2288.
- Ma, Y., Kuijk, R., van Zuylen, H.J., 2013. Day-to-day origin-destination tuple estimation and prediction with hierarchical Bayesian networks using multiple data sources. *Transp. Res. Rec.* 2343 (1), 51–61.
- Mayer, H.M., 1961. Final report, Chicago area transportation study. *Econ. Geogr.* 37 (4), 373–374. <https://doi.org/10.2307/142001>
- Mitsakis, E., Papanikolaou, A., Ayfadopoulou, G., Salanova, J., Doll, C., Giannopoulos, G., Zerefos, C., 2014. An integrated framework for linking climate change impacts to emergency adaptation strategies for transport networks. *Eur. Transp. Res. Rev.* 6, 103–111.
- Mo, B., Ma, Z., Koutsopoulos, H.N., Zhao, J., 2020a. Assignment-based path choice estimation for metro systems using smart card data. *arXiv preprint arXiv:2001.03196*.
- Mo, B., Ma, Z., Koutsopoulos, H.N., Zhao, J., 2020b. Capacity-constrained network performance model for urban rail systems. *Transp. Res. Rec.* 2674 (5), 59–69.
- Mo, B., Ma, Z., Koutsopoulos, H.N., Zhao, J., 2021. Calibrating path choices and train capacities for urban rail transit simulation models using smart card and train movement data. *J. Adv. Transp.* 2021 (1), 5597130.
- Mo, B., Ma, Z., Koutsopoulos, H.N., Zhao, J., 2023. Ex post path choice estimation for urban rail systems using smart card data: an aggregated time-space hypernetwork approach. *Transp. Sci.* 57 (2), 313–335.
- Noursalehi, P., Koutsopoulos, H.N., Zhao, J., 2022. Dynamic origin-destination prediction in urban rail systems: a multi-resolution spatio-temporal deep learning approach. *IEEE Trans. Intell. Transp. Syst.* 23 (6), 5106–5115. <https://doi.org/10.1109/tits.2020.3047047>
- Peng, W., Teng, J., Wang, H., 2024. Understanding heterogeneous passenger route choice in municipal rail transit with express and local trains: an empirical study in Shanghai. *Urban Rail Transit* 10 (2), 122–143.
- Roos, J., Bonneval, S., Gavin, G., 2016. Short-term urban rail passenger flow forecasting: a dynamic Bayesian network approach. In: *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, pp. 1034–1039.
- Roos, J., Gavin, G., Bonneval, S., 2017. A dynamic Bayesian network approach to forecast short-term urban rail passenger flows with incomplete data. *Transp. Res. Procedia* 26, 53–61.
- Sapankevych, N.I., Sankar, R., 2009. Time series prediction using support vector machines: a survey. *IEEE Comput. Intell. Mag.* 4 (2), 24–38.
- Seo, Y., Defferrard, M., Vandergheynst, P., Bresson, X., 2018. Structured sequence modeling with graph convolutional recurrent networks. In: *Neural Information Processing: 25th International Conference, ICONIP 2018, Siem Reap, Cambodia, December 13–16, 2018, Proceedings, Part I* 25. Springer, pp. 362–373.
- Sha, S., Li, J., Zhang, K., Yang, Z., Wei, Z., Li, X., Zhu, X., 2020. Rnn-based subway passenger flow rolling prediction. *IEEE Access* 8, 15232–15240.
- Shi, X., Chen, Z., Wang, H., Yeung, D.-Y., Wong, W.-K., Woo, W.-c., 2015. Convolutional LSTM network: a machine learning approach for precipitation nowcasting. *Adv. Neural Inf. Process. Syst.* 28, 802–810.
- Smith, B.L., Demetsky, M.J., 1997. Traffic flow forecasting: comparison of modeling approaches. *J. Transp. Eng.* 123 (4), 261–266.
- Su, G., Si, B., Zhi, K., Zhao, B., Zheng, X., 2023. Simulation-based method for the calculation of passenger flow distribution in an urban rail transit network under interruption. *Urban Rail Transit* 9 (2), 110–126.
- Sun, Y., Leng, B., Guan, W., 2015. A novel wavelet-SVM short-time passenger flow prediction in Beijing subway system. *Neurocomputing* 166, 109–121.
- Toqué, F., Côme, E., El Mahrsi, M.K., Oukhellou, L., 2016. Forecasting dynamic public transport origin-destination matrices with long-short term memory recurrent neural networks. In: *2016 IEEE 19th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, pp. 1071–1076.
- Tsekeris, T., Dimitriou, L., Stathopoulos, A., 2007. Simultaneous origin-destination matrix estimation in dynamic traffic networks with evolutionary computing. In: *Applications of Evolutionary Computing: EvoWorkshops 2007: EvoCoMnet, EvoFIN, EvoIASP, EvoINTERACTION, EvoMUSART, EvoSTOC and EvoTransLog*. Proceedings. Springer, pp. 668–677.
- Vlahogianni, E.I., Karlaftis, M.G., Golias, J.C., 2014. Short-term traffic forecasting: where we are and where we're going. *Transp. Res. C Emerg. Technol.* 43 (1).
- Wang, S., Miao, H., Chen, H., Huang, Z., 2020. Multi-task adversarial spatial-temporal networks for crowd flow prediction. In: *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pp. 1555–1564.
- Wang, X., Zhang, N., Zhang, Y., Shi, Z., 2018. Forecasting of short-term metro ridership with support vector machine online model. *J. Adv. Transp.* 2018, 3189238. <https://doi.org/10.1155/2018/3189238>
- Wang, Y., Yin, H., Chen, H., Wo, T., Xu, J., Zheng, K., 2019. Origin-destination matrix prediction via graph convolution: a new perspective of passenger demand modeling. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1227–1235.
- Wei, Y., Chen, M.-C., 2012. Forecasting the short-term metro passenger flow with empirical mode decomposition and neural networks. *Transp. Res. C Emerg. Technol.* 21 (1), 148–162.
- Williams, I., 1976. A comparison of some calibration techniques for doubly constrained models with an exponential cost function. *Transp. Res.* 10 (2), 91–104.
- Xi, J., Fei-fan, J., Jia-ping, F., 2018. An online estimation method for passenger flow OD of urban rail transit network by using AFC data. *J. Transp. Syst. Eng. Inf. Technol.* 18 (5), 129.
- Xiong, X., Ozbay, K., Jin, L., Feng, C., 2020. Dynamic origin-destination matrix prediction with line graph neural networks and Kalman filter. *Transp. Res. Rec.* 2674 (8), 491–503.
- Xu, M., Wu, J., Liu, M., Xiao, Y., Wang, H., Hu, D., 2019. Discovery of critical nodes in road networks through mining from vehicle trajectories. *IEEE Trans. Intell. Transp. Syst.* 20 (2), 583–593. <https://doi.org/10.1109/tits.2018.2817282>
- Yang, J., Han, X., Ye, T., Tang, Y., Feng, W., Wang, A., Zuo, H., Zhang, Q., 2022. Spatiotemporal virtual graph convolution network for key origin-destination flow prediction in metro system. *Math. Probl. Eng.* 2022, 5622913. <https://doi.org/10.1155/2022/5622913>
- Yao, H., Tang, X., Wei, H., Zheng, G., Li, Z., 2019. Revisiting spatial-temporal similarity: a deep learning framework for traffic prediction. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 33, pp. 5668–5675.
- Yao, X.-m., Zhao, P., Yu, D.-d., 2015. Real-time origin-destination matrices estimation for urban rail transit network based on structural state-space model. *J. Cent. South Univ.* 22 (11), 4498–4506.
- Yao, X.M., Zhao, P., Yu, D.D., 2016. Dynamic origin-destination matrix estimation for urban rail transit based on averaging strategy. *J. Jilin Univ.* 46 (1), 92–99.
- Yu, B., Yin, H., Zhu, Z., 2017. Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting. *arXiv preprint arXiv:1709.04875*.
- Zhang, J., Che, H., Chen, F., Ma, W., He, Z., 2021. Short-term origin-destination demand prediction in urban rail transit systems: a channel-wise attentive split-convolutional neural network method. *Transp. Res. C Emerg. Technol.* 124. <https://doi.org/10.1016/j.trc.2020.102928>
- Zhang, J., Chen, F., Cui, Z., Guo, Y., Zhu, Y., 2020. Deep learning architecture for short-term passenger flow forecasting in urban rail transit. *IEEE Trans. Intell. Transp. Syst.* 22 (11), 7004–7014.

- Zhang, J., Chen, F., Wang, Z., Liu, H., 2019a. Short-term origin-destination forecasting in urban rail transit based on attraction degree. *IEEE Access* 7, 133452–133462.
- Zhang, J., Li, H., Zhang, S., Yang, L., Jin, G., Qi, J., 2023a. A spatiotemporal graph generative adversarial networks for short-term passenger flow prediction in urban rail transit systems. *Int. J. Gen. Syst.* 52 (6), 694–721.
- Zhang, J., Zheng, Y., Qi, D., 2017. Deep spatio-temporal residual networks for citywide crowd flows prediction. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 31.
- Zhang, J., Zheng, Y., Qi, D., Li, R., Yi, X., Li, T., 2018. Predicting citywide crowd flows using deep spatio-temporal residual networks. *Artif. Intell.* 259, 147–166.
- Zhang, J., Zheng, Y., Sun, J., Qi, D., 2019b. Flow prediction in spatio-temporal networks based on multitask deep learning. *IEEE Trans. Knowl. Data Eng.* 32 (3), 468–478.
- Zhang, S., Zhang, J., Yang, L., Chen, F., Li, S., Gao, Z., 2024. Physics guided deep learning-based model for short-term origin-destination demand prediction in urban rail transit systems under pandemic. *Engineering* 41, 276–296.
- Zhang, Y., Sun, K., Wen, D., Chen, D., Lv, H., Zhang, Q., 2023b. Deep learning for metro short-term origin-destination passenger flow forecasting considering section capacity utilization ratio. *IEEE Trans. Intell. Transp. Syst.* 24 (8), 7943–7960. <https://doi.org/10.1109/tits.2023.3266371>
- Zhao, L., Song, Y., Zhang, C., Liu, Y., Wang, P., Lin, T., Deng, M., Li, H., 2019. T-GCN: a temporal graph convolutional network for traffic prediction. *IEEE Trans. Intell. Transp. Syst.* 21 (9), 3848–3858.
- Zhi-jie, C., Bao-hua, M., Yun, B., Qi, X.U., Tong, Z., 2017. Short-term origin-destination estimation for urban rail transit based on multiple temporal scales. *J. Transp. Syst. Eng. Inf. Technol.* 17 (5), 166.
- Zhou, H., Tan, L., Zeng, Q., Wu, C., 2016. Traffic matrix estimation: a neural network approach with extended input and expectation maximization iteration. *J. Netw. Comput. Appl.* 60, 220–232.
- Zong, X., Guo, J., Liu, F., Yu, F., 2025. TSTA-GCN: trend spatio-temporal traffic flow prediction using adaptive graph convolution network. *Sci. Rep.* 15 (1), 13449.
- Zou, L., Wang, Z., Guo, R., 2024. Real-time prediction of transit origin-destination flows during underground incidents. *Transp. Res. C Emerg. Technol.* 163. <https://doi.org/10.1016/j.trc.2024.104622>